



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER SYSTEMS

ÚSTAV POČÍTAČOVÝCH SYSTÉMŮ

3D NEURAL CELLULAR AUTOMATA

3D NEURÁLNÍ CELULÁRNÍ AUTOMATY

BACHELOR'S THESIS

BAKALÁŘSKÁ PRÁCE

AUTHOR

AUTOR PRÁCE

SUPERVISOR

VEDOUCÍ PRÁCE

MICHAL REPČÍK

Ing. KAREL FRITZ,

BRNO 2026

Bachelor's Thesis Assignment



171171

Institut: Department of Computer Systems (DCSY)
Student: **Repčík Michal**
Programme: Information Technology
Title: **3D Neural Cellular Automata**
Category: Artificial Intelligence
Academic year: 2025/26

Assignment:

1. Study the basics of AI and neural networks.
2. Learn the principles of 2D/3D CA and NCA, and review related studies.
3. Implement a basic 3D NCA architecture for simulating structural development.
4. Select a specific task (e.g., reconstruction or regeneration) and train the model.
5. Run experiments with various parameters and configurations.
6. Evaluate results and discuss limitations and potential applications.

Literature:

According to the instructions of the project supervisor.

Requirements for the semestral defence:

Completion of items 1-3.

Detailed formal requirements can be found at <https://www.fit.vut.cz/study/theses/>

Supervisor: **Fritz Karel, Ing.**
Head of Department: Sekanina Lukáš, prof. Ing., Ph.D.
Beginning of work: 1.11.2025
Submission deadline: 13.5.2026
Approval date: 31.10.2025

Abstract

This thesis presents a modular 3D neural cellular automata implementation in Python and PyTorch. The project includes the simulation core, a training runner, a network server for remote control, and a Blender add-on for live visualization. The work focuses on reproducibility, voxel I/O, and practical use during training and observation of volumetric growth.

Abstrakt

Táto bakalárska práca predstavuje modulárny framework pre 3D neurálne celulárne automaty implementovaný v Pythone a PyTorch. Riešenie pozostáva zo samostatného jadra so simuláciou a tréningovou pipeline, z TCP servera sprístupňujúceho tréning vzdialenému klientovi a z doplnku do Blenderu pre voxelizáciu cieľových 3D objektov a živé vykresľovanie rastu vo viewporte.

Keywords

neural cellular automata, 3D voxel modelling, voxelisation, volumetric generation, morphogenesis, metamorphogenesis, conditional generation, interactive visualisation, Blender add-on, PyTorch, TCP server

Klíčové slová

neurálne celulárne automaty, 3D voxelové modelovanie, voxelizácia, volumetrické generovanie, morfogenéza, metamorfogenéza, podmienené generovanie, interaktívna vizualizácia, Blender add-on, PyTorch, TCP server

Reference

REPČÍK, Michal. *3D Neural Cellular Automata*. Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Karel Fritz,

Rozšírený abstrakt

Neurálne celulárne automaty (NCA) sú generatívne modely, v ktorých lokálne pravidlo klasického celulárneho automatu nahrádza malá neurónová sieť zdieľaná všetkými bunkami. Každý voxel aktualizuje svoj stav iba na základe lokálneho okolia, no opakovaním týchto interakcií vzniká komplexné globálne správanie: rast, tvorba štruktúr, stabilizácia tvaru či čiastočná regenerácia. Rozšírenie tohto princípu do troch rozmerov vedie k práci s voxelovými mriežkami a otvára možnosti pre volumetrické modelovanie, procedurálnu generáciu a experimentovanie s rastúcimi 3D objektmi.

Doterajšie 3D implementácie NCA vo väčšine prípadov existujú ako izolované tréningové skripty. Chýba im integrovaný pracovný postup na prípravu cieľov z reálnych 3D objektov, živá vizualizácia rastu, jednoduché prepojenie na vzdialené GPU a najmä infraštruktúra, ktorá by umožnila rýchlo pridávať nové experimenty bez zásahov do množstva nesúvisiacich častí. Pri zmene cieľa, režimu tréningu alebo spôsobu vizualizácie tak používateľ často musí skladať viacero nástrojov bez spoločných rozhraní a bez dobrej reprodukovateľnosti.

Cieľom tejto práce je vytvoriť modulárny framework, ktorý túto medzeru vyplní a umožní praktické experimentovanie s 3D NCA. Navrhnuté riešenie je rozdelené do troch nezávislých vrstiev. Prvou vrstvou je samostatné jadro v Pythone a PyTorch, ktoré obsahuje model, tréningové cykly, plánovač udalostí, logovanie a checkpointy. Toto jadro je použiteľné priamo zo skriptov alebo z Jupyter notebookov bez závislosti na Blendre. Druhou vrstvou je TCP server, ktorý jadro sprístupní cez sieť: prijíma konfiguráciu a cieľové voxelové dáta, spúšťa tréning a priebežne vysiela snímky stavu mriežky. Tretou vrstvou je doplnok do Blenderu, ktorý plní rolu klienta a grafického rozhrania: pripravuje cieľové voxelizáciu mesh objektov (vrátane odberu farby z materiálov) a vykresľuje prijaté stavy ako voxelové modely priamo vo viewporte. Vďaka tejto separácii môže tréning bežať na vzdialenom GPU serveri a Blender môže zostať lokálnym vizualizačným klientom; komunikácia funguje aj cez tunelované spojenie (napr. pomocou ngrok).

Jadro modelu pracuje s 3D mriežkou, kde každý voxel nesie vektor kanálov. Viditeľné kanály reprezentujú veličiny porovnávané s cieľom (typicky RGBA, kde alfa slúži ako signál, ktorý určuje či je voxel živý), skryté kanály slúžia ako vnútorná pamäť bez priamej supervízie. Framework podporuje dve rozšírenia potrebné pre náročnejšie úlohy: pozičné kanály s normalizovanými súradnicami (x, y, z) poskytujú bunkám globálny referenčný rámec pre asymetrické cieľe a task kanály umožňujú podmienenie jedného modelu viacerými cieľmi. Percepcia lokálneho okolia je realizovaná fixnými $3 \times 3 \times 3$ filtrami (identita, súčet susedov a izotropné alebo smerové operátory), čím sa do aktualizáčného pravidla dostane kompaktná reprezentácia okolia bez potreby učiť sa konvolučné filtre. Aktualizačné pravidlo je malý MLP aplikovaný nezávisle na všetkých pozíciách; jeho výstup sa pripočíta k stavu a môže byť navyše riadený stochastickou maskou asynchrónnej aktualizácie. Masky živosti je odvodená z alfa kanála pomocou 3D max-poolingu tak, aby aj mŕtve voxely susediace so živými mohli dostávať nenulové prírastky a rast sa prirodzene šír po hranici štruktúry.

Tréning je formulovaný ako diferenciabilná optimalizácia parametrov aktualizáčného pravidla. Základným režimom je morfogénéza, v ktorej z jedného semienka vyrastá jeden cieľový tvar. Praktickú stabilitu zlepšuje pool-based tréning: namiesto opakovaného resetu na semienko sa udržiava zásobník stavov v rôznych fázach vývoja, z ktorého sa v každej iterácii vzorkuje dávka, rozvíja na náhodný počet krokov a následne sa aktualizované stavy zapisujú späť do poolu. Parametre ako počet krokov, váhy zložiek straty či learning rate sa môžu počas tréningu meniť pomocou plánovača udalostí, takže prechody medzi fázami nie je potrebné natvrdo kódovať do tréningovej slučky.

Nad rámec morfogénzy práca zavádza a demonštruje metamorfogézu: viaccieľový režim, v ktorom jeden model dokáže vytvárať viac stabilných tvarov a plynulo medzi nimi prechádzať prepínaním task kanálov. Tréning používa zdieľaný replay pool pre všetky úlohy naraz; na hraniciach fáz sa pool rozšíri tak, že najlepšie konvergované stavy z predchádzajúcej úlohy sa skopírujú a prešitkujú na ďalšiu úlohu. Nová fáza tak nezačína z náhodného semienka, ale z realistických medzistavov, čo podporuje prirodzené prechody. Pri inferencii potom stačí prepísať one-hot embedding task kanálov a systém pokračuje v raste smerom k ďalšiemu cieľu bez re-seedovania.

Framework je overený experimentmi na mriežke 32^3 . Symetrická morfogéza potvrdila stabilný rast aj stabilitu po skončení tréningového horizontu a zároveň prakticky otestovala plnú interaktívnu cestu, kde tréning bežal na vzdialenom GPU a Blender zobrazoval priebeh v reálnom čase cez tunelované TCP spojenie. Asymetrická morfogéza na voxelizovanom šachovom pešiakovi odhalila limitáciu čisto izotropného vnímania: model mal tendenciu generovať symetrické objekty. Zapnutie smerových gradientov a doplnenie pozíčných kanálov poskytlo orientačný signál a umožnilo zachovať globálnu orientáciu cieľa. Metamorfogéza bola najprv validovaná na syntetickom trojcieľovom pláne (ihlan $\rightarrow$ kocka $\rightarrow$ guľa), kde sa ukázalo, že samotný prepis task kanálov stačí na kontrolované prechody. Následne bola rovnaká technika aplikovaná na tri komplexnejšie farebné ciele pripravené v Blenderi, čo demonštrovalo použiteľnosť aj pri komplexnejšej geometrii a farbách; najnáročnejšie zostali tenké a vysoko asymetrické časti pri obmedzenom rozlíšení 32^3 .

Výsledkom práce je použiteľný, reprodukovateľný a rozšíriteľný súbor nástrojov pre 3D neurálne celulárne automaty. Architektúra umožňuje, aby nový experiment vyžadoval iba implementáciu alebo úpravu jednej triedy, pričom server, sieťový protokol a Blender vizualizácia zostávajú bez zmeny. Metamorfogéza s task kanálmi a fázovým rozširovaním poolu navyše demonštruje, že jedno lokálne aktualizčné pravidlo môže kódovať viacero 3D tvarov a riadené prechody medzi nimi. Budúca práca môže rozšíriť experimenty regenerácie, zvýšiť rozlíšenie mriežky a zlepšiť reprezentáciu detailných štruktúr, ktoré sú pri voxelových cieľoch najcitlivejšie.

3D Neural Cellular Automata

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Ing. Karel Fritz I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used Chat-GPT and Grammarly to refine the text and correct spelling and other language mistakes. I used Copilot and Claude Code when working on the software.

.....

Michal Repčík
May 7, 2026

Acknowledgements

I would like to express my sincere thanks to my supervisor, Ing. Karel Fritz, for his guidance and support.

Contents

1	Introduction	5
2	Abstract	7
3	Theoretical Foundations	8
3.1	Cellular Automata	8
3.2	Neural Cellular Automata	10
3.3	Extension from Two to Three Dimensions	11
3.4	Core Components of a 3D Neural Cellular Automaton	12
4	Related Work	15
4.1	Growing Neural Cellular Automata	15
4.2	Volumetric NCA in Interactive Environments	17
4.3	Prior Open-Source Implementations	18
4.4	Chapter Conclusion	18
5	Architectural Design	19
5.1	Design Goals and Requirements	19
5.2	System Overview	20
5.3	Core NCA Model	21
5.4	Training Pipeline	23
5.5	Server and Networking	24
5.6	Blender Integration	27
6	Implementation	29
6.1	Core Framework	29
6.2	NCA Model Components	30
6.3	Training Pipeline	34
6.4	Server	35
6.5	Blender Add-on	38
7	Testing	39
7.1	Automated Tests	39
7.2	Protocol Verification	40
7.3	Notebook Experiments	40
7.4	End-to-End Testing	40
8	Experiments	41
8.1	Morphogenesis	42

8.2 Metamorphogenesis	49
9 Conclusion	60
Bibliography	61

List of Figures

5.1	High-level system architecture. Arrows indicate dependency direction. The dashed TCP boundary marks the network separation between the server and the Blender client.	20
5.2	Per-voxel state vector layout. The vector contains hidden channels, optional positional channels, optional task channels and visible channels. The alpha channel, which serves as the alive signal, is always the last visible channel. .	21
5.3	Data flow through a single NCA update step.	23
5.4	Client-server message sequence for a complete session. The client drives all state transitions; the server responds with acknowledgements and streams state broadcasts during active training or inference.	25
5.5	Session state machine. The server transitions between disconnected, idle, training, paused and inference states in response to client messages. All active states return to disconnected if the client closes the connection. . . .	26
6.1	Detailed data flow through a single NCA update step, showing the full layer sequence within the update MLP, the optional stochastic fire mask and the bounded output. The diagram uses the default three-group perception configuration; the directional five-group variant follows the same structure. . .	33
8.1	Symmetric target used for baseline morphogenesis.	43
8.2	Symmetric morphogenesis from a single seed over inference steps.	43
8.3	Final state at epoch 3000 compared with the symmetric target.	44
8.4	Loss curves for symmetric morphogenesis training.	44
8.5	Failed asymmetric pawn run at epoch 5000 with symmetric perception configuration. The prediction overgrows the target and collapses into repeated symmetric structures.	45
8.6	Asymmetric target prepared in Blender (top) and voxel target used during training (bottom).	46
8.7	Asymmetric morphogenesis from a single seed over inference steps.	47
8.8	Final asymmetric output at epoch 4000.	47
8.9	Loss curves for asymmetric morphogenesis training.	48
8.10	Synthetic targets for the three-target curriculum: tetrahedron, cube and sphere.	50
8.11	Loss curves for the synthetic curriculum. Dashed vertical lines mark phase boundaries at epochs 3000 and 6000.	51
8.12	Inference progression for the synthetic curriculum. Each row continues from the previous row's final state with no re-seeding – only the task-channel embedding is overwritten.	51
8.13	Target versus model output after synthetic curriculum training.	52

8.14	Blender-exported targets for the creature sequence: Creature 1, Creature 2 and Creature 3.	53
8.15	Loss curves for Creature Round 1 (9k epochs). Dashed lines mark phase boundaries at epochs 3000 and 6000.	54
8.16	Target versus model output for Creature Round 1.	54
8.17	Loss curves for Creature Round 2 (12k epochs). Phase boundaries at epochs 2001 and 7001.	56
8.18	Loss curves for Creature Round 3 (19.5k epochs). Phase boundaries at epochs 1501 and 7501.	56
8.19	Target versus model output for Creature Round 2 (12k epochs).	57
8.20	Target versus model output for Creature Round 3 (19.5k epochs).	58
8.21	Inference progression for Creature Round 3. Each row continues from the previous row's final state; only the task-channel embedding is overwritten between rows.	59

Chapter 1

Introduction

Neural cellular automata are a class of generative models in which a small shared neural network replaces the hand-crafted transition rule of a classical cellular automaton. Each cell updates its state based only on its local neighbourhood, yet complex global structure – growth, pattern formation, regeneration – emerges from these repeated local interactions without any centralised coordination. Extending this paradigm to three-dimensional voxel grids is conceptually straightforward and opens practical applications in volumetric modelling and procedural content generation.

Prior work has confirmed that three-dimensional growth of this kind is achievable. What it has not produced is usable tooling. Existing implementations are mostly standalone training scripts with no live visualisation, no way of interacting with the model during training and no practical path from a real mesh to a ready-to-use coloured voxel dataset. A researcher wanting to try a new target or a different model configuration ends up piecing together separate tools with no shared infrastructure between them.

The goal of this thesis is to provide the infrastructure that would enable researchers or people interested in NCA to easily experiment with different configurations and targets. We designed and implemented a modular framework consisting of three decoupled components: a Blender add-on for mesh voxelisation and live viewport rendering; a TCP server that exposes any training backend over a local or tunnelled connection; and a standalone Python core package providing the NCA model and training pipeline independently of both. The central design requirement was that adding a new experiment should require writing one class, with the server, protocol, logging and visualisation pipeline requiring no changes. The metamorphogenesis experiments described later confirm this in practice – they required only a custom runner subclass, minimal changes to the core module and no changes to the server or visualisation components.

To reach this goal, the work proceeded through four steps: reviewing existing implementations to understand where the tooling gaps actually were; designing the three-layer architecture with well-defined boundaries between components; implementing the core model with the positional and task-channel extensions needed for asymmetric and multi-target settings; and running morphogenesis and metamorphogenesis experiments to validate the complete system.

The rest of the thesis is organised as follows. The Theoretical Foundations chapter covers the background from classical and continuous cellular automata through to the neural CA formulation the framework builds on. The Related Work chapter examines the original 2D Growing NCA and its subsequent three-dimensional extensions, identifying the limitations that motivated this project. Architectural Design and Implementation describe the

system structure and how its components communicate. Testing covers the automated unit and integration suite. The Experiments chapter presents the morphogenesis and metamorphogenesis results and serves as the primary end-to-end validation of the framework. The Conclusion summarises the outcomes and outlines directions for future work.

Chapter 2

Abstract

Existing 3D Neural Cellular Automata implementations are mostly standalone training scripts with no interactive tooling and no integrated workflow for preparing mesh targets or exporting coloured voxel datasets. This thesis presents a modular framework consisting of a Blender add-on for mesh voxelisation and live viewport rendering, a TCP server that exposes any training backend over a local or tunnelled connection and a standalone Python core package usable in Jupyter notebooks without Blender. The framework is validated through morphogenesis experiments on symmetric and asymmetric targets and through multi-target metamorphogenesis, where a single model grows and transitions between distinct voxel shapes conditioned on task channels. The primary contributions are the modular framework itself, in which implementing a new experiment requires subclassing one abstract class with no changes to the server, protocol, logging, or visualisation pipeline and the metamorphogenesis training regime, which demonstrates task-conditioned multi-target growth in 3D in a way not previously explored in the literature.

Chapter 3

Theoretical Foundations

This chapter introduces all concepts relating to cellular automata and their neural extensions from scratch, starting from classical discrete systems and working up to the fully differentiable neural CA formulation the framework is built on.

3.1 Cellular Automata

Cellular automata (CA) are discrete dynamical systems composed of a regular grid of cells, each occupying one state from a finite set. All cells update in parallel at each discrete time step according to a local rule that depends only on the states of neighbouring cells [12].

A cellular automaton is defined by:

- a **lattice** of cells, typically arranged in one, two, or three dimensions,
- a **finite state set** $\mathcal{S}$,
- a **neighbourhood function** $\mathcal{N}$ describing which cells influence a given cell,
- an **update rule** $f : \mathcal{S}^{|\mathcal{N}|} \rightarrow \mathcal{S}$ that determines the next state [8].

Ulam and von Neumann developed the first formal CA models in the 1940s as a theoretical framework for self-replicating machines [15]. Von Neumann’s original construction used a two-dimensional lattice with 29 distinct cell states and demonstrated that a sufficiently complex local rule can produce a system capable of constructing copies of itself without any centralised coordination.

Wolfram later classified CA dynamics into four behavioural classes [24, 25]:

- **Class 1** – the system converges to a uniform state regardless of initial conditions,
- **Class 2** – activity settles into static or periodic patterns,
- **Class 3** – evolution is chaotic and sensitive to initial conditions,
- **Class 4** – the system produces complex, long-lived interacting structures capable of supporting computation.

This classification reappears throughout the chapter as a reference point for characterising the dynamics of both classical and neural CA.

3.1.1 One-Dimensional Cellular Automata

The simplest non-trivial CA operate on a one-dimensional array of cells. In the elementary CA family, each cell reads its own state and its two immediate neighbours (radius 1, neighbourhood size 3). With binary states, $2^3 = 8$ neighbourhood configurations are possible, giving $2^8 = 256$ distinct rules in Wolfram’s enumeration [24].

Each rule number encodes a next-state lookup table in binary. **Rule 90** assigns the next state of a cell based on the exclusive-or (XOR) of its two neighbours, producing the Sierpiński triangle from a single active seed – a fractal, self-similar pattern that places Rule 90 in Wolfram’s **Class 3**. **Rule 30** also belongs to Class 3 and produces irregular, aperiodic evolution that has been used as a source of pseudorandomness [25].

Rule 110 (Class 4) is the smallest known Turing-complete one-dimensional CA. Cook [4] proved that it supports gliders and glider collisions capable of simulating any Turing machine, showing that computational universality can arise from a simple local rule on a one-dimensional lattice.

Despite their simplicity, elementary CA exhibit all four classes of Wolfram’s classification – a minimal demonstration of how local rules can produce fundamentally different global behaviour. The same question extends directly to the neural CA models discussed later in this chapter.

3.1.2 Two-Dimensional Cellular Automata

In two dimensions, the Moore neighbourhood (3×3 block surrounding each cell) is the most common choice. With binary states and a Moore neighbourhood, the update rule must handle $2^9 = 512$ possible input configurations. Because the full lookup table is large, 2D rules are typically expressed using birth/survival notation. A rule written as Bn/Sm specifies that a dead cell becomes alive (is “born”) when it has exactly n live neighbours and a living cell survives when it has exactly m live neighbours. All other cells die or remain dead.

Conway’s Game of Life [6] uses the rule B3/S23: a dead cell becomes alive with exactly three live neighbours and a living cell survives with two or three. Despite this simplicity, Life produces a wide variety of emergent structures: stable **still lifes** such as the block and beehive; **oscillators** such as the blinker and toad, which cycle through a fixed sequence of states; **spaceships** such as the glider, which translate across the grid while cycling through their phases; and more complex constructions such as glider guns and computational logic structures.

Berlekamp, Conway and Guy [1] showed that Life is Turing-complete: any computation expressible as a Turing machine can, in principle, be carried out within a sufficiently large Life grid. This is a clear illustration of the defining property of cellular automata – complex, globally coordinated behaviour emerging from a homogeneous local rule – and the same property later motivated the development of neural cellular automata, where the local rule is learned rather than hand-designed.

3.1.3 Three-Dimensional Cellular Automata

In three dimensions, the Moore neighbourhood covers 26 adjacent cells (6 face-adjacent, 12 edge-adjacent and 8 corner-adjacent) in a $3 \times 3 \times 3$ cube surrounding the central cell. Binary states with a full Moore neighbourhood produce $2^{27} \approx 134$ million neighbourhood configurations, making complete rule enumeration impractical. Practical 3D CA therefore

use totalistic or outer-totalistic rules, where the update depends only on the count of live neighbours rather than their exact arrangement [25].

Several notable 3D CA have been studied. Von Neumann’s 29-state self-replicating automaton [15], originally formulated in 2D, has been extended to three dimensions, where the additional spatial freedom allows more compact replicating structures. Excitable-media rules such as Brian’s Brain have also been generalised to 3D lattices, producing travelling wavefronts and spiral structures [10]. Three-dimensional Game of Life variants using totalistic birth/survival rules generate branching and crystal-like growth patterns that have no direct 2D analogue.

GPU-accelerated implementations can run volumetric CA with grid sizes exceeding 128^3 cells in real time [22], making large-scale 3D CA simulation practical on consumer hardware. This same lattice structure and local-update approach carries over directly to the neural CA extensions discussed later in this chapter.

3.1.4 Differentiability and Continuous Cellular Automata

Classical CA use discrete states and discrete update rules. Several extensions have shown that the core CA principles – a spatial lattice, local interactions and parallel updates – also work with continuous states and smooth transition functions. These continuous CA are directly relevant to neural CA, which operate on real-valued state vectors and use differentiable update rules trained by gradient descent.

An early theoretical connection was established by Turing [23], whose reaction-diffusion model of morphogenesis describes how spatial patterns arise from the interaction of locally diffusing chemical substances. Turing’s model uses partial differential equations rather than a discrete lattice, but the underlying principle – local interactions producing global spatial structure – is shared with cellular automata and it has since motivated much of the work on continuous and neural CA.

SmoothLife [18] generalises Conway’s Game of Life to continuous space and time. It replaces the discrete neighbour count with continuous averaging kernels and the binary birth/survival rule with smooth sigmoid transition functions, producing gliders, oscillators and wave-like structures that closely resemble their discrete counterparts but evolve smoothly in time. The characteristic behaviours of Life, it turns out, do not depend on discretisation – they arise from the structure of local interactions.

Lenia [2] extends this further by parameterising both the convolution kernel and the growth function. Systematic exploration of its parameter space has produced hundreds of stable, self-organising patterns that resemble biological microorganisms – orbiting particles, pulsating organisms, multi-species ecologies.

Both systems show that complex self-organisation does not require discrete state transitions. Neural cellular automata take the next step: rather than manually exploring the parameter space as in Lenia, neural CA use gradient-based optimisation to find update rules that produce a specific target pattern. The differentiability of the update rule is what makes this possible.

3.2 Neural Cellular Automata

Mordvintsev et al. [13] introduced Neural Cellular Automata (NCA) as a differentiable generalisation of classical CA. The discrete lookup-table rule is replaced by a small neural network whose parameters are shared across all cells. The spatial lattice stays discrete, but

each cell’s state becomes a real-valued vector, and the update rule is trained end-to-end by gradient descent.

Let the lattice be $\Omega \subset \mathbb{Z}^d$, where d is the spatial dimension. Each cell $x \in \Omega$ holds a state vector

$$\mathbf{s}_t(x) \in \mathbb{R}^C,$$

where C is the number of channels. The full grid state at time t is a tensor over all cell states.

The update process has two stages [13]. First, a fixed perception operator extracts local neighbourhood information through convolutional filters applied to the full multi-channel grid, producing a perception vector at each cell location. Second, a shared neural network maps this perception vector to a state increment $\Delta \mathbf{s}$ added to the current state:

$$\mathbf{s}_{t+1}(x) = \mathbf{s}_t(x) + \Delta \mathbf{s}(x).$$

This residual formulation preserves the defining CA property – strict locality – while replacing the hand-crafted transition function with a learnable model. Because the update rule is differentiable with respect to its parameters, the automaton can be unrolled over multiple time steps and trained by gradient descent to minimise a loss defined on the resulting configuration.

The key shift compared to classical and continuous CA is that the update rule is not designed or discovered by exploration, but optimised for a specific objective. The system learns to grow a target pattern from a seed, maintain it over time, or regenerate it after damage, depending on the training regime. Global behaviour still emerges from distributed local interactions – but the local rule itself is shaped by optimisation.

3.3 Extension from Two to Three Dimensions

The original NCA of Mordvintsev et al. [13] operates on a 2D grid. Extending it to three-dimensional voxel lattices is conceptually straightforward but introduces practical differences that affect both the architecture and the training procedure.

In 2D, perception uses 3×3 filters over a nine-cell Moore neighbourhood. In 3D, the same filters generalise to $3 \times 3 \times 3$ volumes covering twenty-seven cells. Prior 3D extensions have shown that Sobel-type gradient filters and other perception operators transfer directly to x , y and z axes without changing the overall update formulation [21, 19, 9].

Two practical differences are worth noting. First, cell count grows cubically with resolution: a 32^3 grid contains 32 768 cells, while a 64^3 grid contains 262 144. This scaling increases both memory consumption and computational cost per step, particularly during training when gradients must be stored for backpropagation.

Second, information propagates only through local interactions, one step at a time. In a 32^3 grid, a signal originating at one corner must traverse roughly 55 steps to reach the opposite corner. The model must therefore remain stable over longer simulation horizons to produce coherent global structure, compared to a 2D system of similar resolution.

How this affects specific behaviours – persistence, regeneration, long-term stability – is not fully established in the literature. The neighbourhood grows from 9 to 27 cells, the propagation distances increase and the spatial topology changes. Whether the dynamics observed in 2D carry over cleanly to volumetric grids is one of the questions motivating the experiments in this thesis.

3.4 Core Components of a 3D Neural Cellular Automaton

This section describes the main components of a three-dimensional neural cellular automata at the conceptual level, establishing the terminology and design principles referenced throughout the rest of the thesis.

3.4.1 State Representation

Each voxel in a 3D neural CA maintains a vector of C real-valued channels:

$$\mathbf{s} \in \mathbb{R}^C.$$

The channels divide into two functional groups. **Visible channels** represent quantities compared directly to a target pattern, such as colour components (RGBA) or occupancy. **Hidden channels** carry internal memory that is not directly supervised but passes information between neighbouring cells across time steps [13].

This separation is a design choice inherited from the original 2D NCA formulation. Because updates are strictly local, a cell cannot observe distant parts of the grid directly. Hidden channels give each cell the capacity to accumulate and relay information over multiple steps, which is what allows coherent global patterns to emerge from purely local rules.

Most NCA models also designate one channel as a continuous *alive* signal, determining whether a cell is considered active and whether its update is applied. This mechanism appears in the original NCA [13] and has parallels in continuous CA such as SmoothLife [18] and Lenia [2], where real-valued fields govern the activation of spatial regions.

3.4.2 Perception Mechanism

Neural CA preserve the locality constraint that defines classical CA: each cell updates based only on its immediate neighbourhood [8, 25]. In a 3D neural CA, this is realised by applying a fixed set of convolutional filters with kernel size $3 \times 3 \times 3$ to the multi-channel state grid, producing a perception vector at each cell location [13, 21].

Common filter types include:

- **Identity filters**, which return the cell’s own state unchanged, providing the update rule with information about what the cell currently contains.
- **Neighbour aggregation filters**, which compute a sum or average of adjacent voxels, capturing the local density and composition of the neighbourhood.
- **Gradient filters**, such as Sobel-type kernels extended to the x , y and z axes, which approximate directional differences across the neighbourhood [13, 21].

The outputs of all filters applied across all channels are concatenated into a single perception vector per cell. With C channels and G filter groups, the perception vector has dimensionality $G \cdot C$, which is then passed to the update rule.

The filters are fixed and non-learnable. This constrains the spatial information available to the update rule, stabilises training and reduces the number of parameters to optimise – a convention established by Mordvintsev et al. [13] and followed in subsequent work [21, 19, 9].

3.4.3 Update Rule

The update rule is a shared parametric function that maps each cell’s perception vector to a state increment $\Delta\mathbf{s}$, added residually to the current state:

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \Delta\mathbf{s}.$$

By using a residual update, the model makes small, incremental changes instead of sudden leaps. This approach is essential for maintaining stability when the model is unrolled over many many steps during training [13].

The update function is implemented as a small multilayer perceptron (MLP) applied independently at each cell, ensuring that each increment depends only on locally perceived information. The parameter count is deliberately kept low – approximately 8 000 trainable parameters in the original 2D model [13]. The complexity of the resulting structures comes from iterative local interaction, not from network capacity.

Two stabilisation mechanisms are standard in NCA formulations:

- **Alive gating** conditions updates on the alive channel. If a cell and all of its neighbours have alive values below a threshold, the cell’s update is set to zero, preventing empty regions from spontaneously developing structure [13].
- **Stochastic update masking** applies updates to a random subset of cells at each step rather than all cells simultaneously, eliminating global synchronisation errors and ensuring system stays stable even under asynchronous updates [13].

Both mechanisms improve long-term stability and are particularly relevant in 3D, where instabilities can accumulate across a larger number of cells.

3.4.4 Grid Management

The neural CA runs on a finite three-dimensional lattice with fixed dimensions. Boundary conditions – periodic, zero-padded, or reflective – are chosen per experiment and affect dynamics near the grid edges [13]. Sudhakaran et al. [21] note that boundary handling varies across implementations and can influence the shapes the automaton produces.

The primary practical challenge of 3D grids is cubic memory scaling. A grid of size N^3 with C channels per cell stored at 32-bit precision requires $N^3 \times C \times 4$ bytes per state. For a 32^3 grid with 20 channels this is approximately 2.5 MB; during training, multiple states are held simultaneously for batching and pool-based sampling and gradient storage adds further overhead.

Sparse data structures address this at larger scales. Sparse Voxel Octrees [11] store only occupied regions in a hierarchical tree. OpenVDB [14] provides a similar approach with dynamic topology. The framework developed in this thesis uses dense grids at moderate resolution, but these representations offer a natural path toward scaling to larger volumetric domains.

Most neural CA are initialised from a *seed* – a single active cell placed at the centre of an otherwise empty grid – from which the system must grow the target structure through repeated application of the local update rule.

3.4.5 Training and Optimisation

Because the update rule is differentiable, the model parameters can be optimised by gradient descent. The standard procedure unrolls the automaton for T steps from a seed state, computes a loss between the resulting configuration and a target and backpropagates through the full unrolled computation graph [13].

The most common loss function is mean squared error (MSE) between the predicted and target visible channels. The optimiser is typically Adam or one of its variants, with learning rates in the range of 10^{-4} to 10^{-2} [21].

Several training strategies have been developed to encourage specific behaviours:

- **Pool-based sampling** maintains a pool of persistent states at various stages of growth. Each iteration samples a batch from the pool, runs the automaton forward, computes the loss and writes the updated states back, exposing the model to different developmental stages rather than always starting from a fresh seed [13].
- **Curriculum learning** gradually increases the number of unrolled steps during training. Early iterations use short unrolls to learn coarse shape; later iterations use longer unrolls to test stability over extended horizons.
- **Training regime selection** determines which aspects of the dynamics are explicitly optimised. These regimes – growth, persistence, regeneration – are discussed in the Related Work chapter.

Backpropagating through many unrolled steps requires storing intermediate activations for the entire sequence, which can exhaust GPU memory. Gradient checkpointing [17] addresses this by discarding intermediate activations during the forward pass and recomputing them during the backward pass, trading roughly twice the computation for substantially reduced memory usage.

Chapter 4

Related Work

This chapter reviews the research most directly related to neural cellular automata and their three-dimensional extensions, focusing on architectures, training procedures and experimental results. The core NCA components described in Section 3.4 – state representation, perception, update rule and training – were drawn directly from Mordvintsev et al. [13] and are not repeated here. Each section instead focuses on the concrete design decisions, experimental findings and limitations of the reviewed work and on what they mean for the framework developed in this thesis.

4.1 Growing Neural Cellular Automata

Mordvintsev, Randazzo, Niklasson and Levin [13] introduced the Growing Neural Cellular Automata paradigm that serves as the methodological basis for this work. Unlike continuous CA such as Lenia [2], which rely on manual parameter exploration, neural CA use supervised optimisation: the transition function is trained end-to-end to produce a specific target pattern.

The 2020 implementation uses a 16-channel state vector per cell – four visible RGBA channels supervised against a target image and twelve hidden channels acting as distributed memory. Perception is handled by three fixed filter groups (identity, Sobel in x and y , Laplacian) applied across all channels, producing a 48-dimensional perception vector at each location. A small shared MLP with approximately 8000 trainable parameters maps this vector to a residual state increment. Alive gating and stochastic update masking, both introduced in Section 3.4, are applied throughout. The result is a homogeneous system where all behavioural differences between cells emerge from their local spatial context rather than from any privileged role in the architecture.

4.1.1 Training Regimes

Mordvintsev et al. [13] introduced three training regimes that target different aspects of the learned dynamics. The choice of regime determines the balance between initial growth, long-term stability and robustness to damage.

Growing Regime

The model is trained to produce a target from a single seed, with loss computed after a fixed number of steps T . No constraint is imposed beyond T – patterns may drift or diverge if simulation continues past the training horizon. In practice, growing-only models commonly become unstable within 100–200 steps beyond T [13].

Persistent Regime

Loss is evaluated at multiple steps beyond initial convergence – at T , $T + \delta$, $T + 2\delta$ and so on – penalising any drift from the target. This produces more stable dynamics. Persistent models also showed limited regenerative behaviour without any explicit damage training.

Regenerating Regime

Random damage is applied during training by zeroing out rectangular regions of the grid, requiring the model to reconstruct the target from the damaged state. This produces the most robust dynamics, with models recovering from substantial perturbations and maintaining structural integrity over extended horizons [13]. Horvath and Levin [7] later showed that stochastic update masking – one of the stabilisation mechanisms used throughout training – also encourages distinct functional cell types to emerge within a single NCA, as cells must develop local strategies that hold regardless of which neighbours updated in the previous step.

4.1.2 Relevance to This Work

This work adopts the architecture, training regimes and optimisation strategy of Mordvintsev et al. [13] as its baseline and extends them to three-dimensional voxel lattices. The transition from 2D to 3D increases neighbourhood size from 9 to 27 cells, raises computational cost cubically and extends the propagation distances required for global coordination. Whether the stability and regenerative behaviours observed in 2D carry over to volumetric grids is one of the central questions motivating the experiments in this thesis.

4.2 Volumetric NCA in Interactive Environments

Sudhakaran et al. [21] extended NCA to three dimensions in the sandbox game *Minecraft*, evaluating volumetric growth of structures and functional machines under real-time constraints and user interaction.

The model follows the 2D growing NCA paradigm directly. Each voxel stores a state vector with visible and hidden channels; perception uses $3 \times 3 \times 3$ neighbourhoods with identity and gradient features processed by a small shared network. The system is trained by gradient descent to grow a target structure from a single seed.

The experiments confirmed that neural CA can produce coherent volumetric structures in a dynamic, interactive environment. The learned rules remain local and homogeneous throughout, with complex global structure emerging from repeated local updates. The authors also observed partial regenerative behaviour: when parts of a built structure were manually destroyed, the NCA could partially rebuild the damaged region.

An interesting aspect of this work is its evaluation under real-world constraints. The *Minecraft* platform fixes the available block types, rendering pipeline and interaction tools – all of which differ from the unconstrained continuous-valued grids used in purely computational settings. That the NCA paradigm still produced recognisable structures under these conditions shows the approach is not limited to idealised simulation environments.

4.2.1 Experimental Scope and Constraints

The evaluation is primarily a proof of concept: the experiments demonstrate successful growth toward predefined targets but stop short of systematic analysis. There are no measurements of long-term stability, no controlled damage experiments and no comparison against other 3D generative methods. The *Minecraft* environment contributes to this limitation. Discrete block types, a fixed renderer and game-based interaction tools all constrain how precisely experiments can be set up and measured.

4.2.2 Relation to This Work

Sudhakaran et al. [21] provide the primary basis for extending neural CA from planar grids to volumetric $3 \times 3 \times 3$ perception neighbourhoods. The core principles – strict locality, parameter sharing and gradient-based optimisation – carry over directly into the framework developed here.

The main difference is the environment. This work uses Blender rather than a game engine, which gives direct programmatic access to the voxel grid, mesh representations, rendering pipelines and the Python scripting API. This makes it practical to vary grid resolution, boundary conditions and material properties systematically – aspects that the *Minecraft* setting does not expose to the researcher. It also opens the door to future extensions that would be difficult in a game environment, such as integrating more complex mesh targets, alternative voxelisation strategies, or new training regimes. Where Sudhakaran et al. showed that 3D NCA works in an interactive environment, the goal here is to provide the tooling for controlled experimentation and to build a foundation that can be extended further.

4.3 Prior Open-Source Implementations

Beyond the work of Mordvintsev et al. [13] and Sudhakaran et al. [21], several open-source projects have explored three-dimensional neural CA. This section reviews two implementations whose design decisions are relevant to the framework developed in this thesis.

4.3.1 Single-Target 3D Reconstruction

The repository of Aadityaza [9] implements a direct extension of the Growing NCA paradigm to three-dimensional space using a Jupyter Notebook environment and PyTorch, reconstructing voxel shapes from a single seed.

The project confirms that the convolution-based perception approach from 2D NCA transfers directly to volumetric tensors – specifically, that Sobel-type filters extend cleanly to all three spatial axes without losing locality. For this work it served as an early reference point for validating the feasibility of the 3D extension.

The implementation covers single-target reconstruction only. There is no training server, no interactive visualisation and no modular structure for swapping model components. These are gaps the framework developed in this thesis addresses directly.

4.3.2 Voxel-Based Morphogenesis

Rossi and collaborators [19] extended the 2D growing NCA to volumetric grids with $3 \times 3 \times 3$ perception filters and hidden channel representations, training the system to approximate 3D voxel targets from simple seeds.

The architecture follows the 2D formulation closely – fixed perception filters, residual updates, shared parameters, gradient-based optimisation – and the experiments confirm that volumetric growth works under strict locality constraints. The evaluation is limited to demonstrating successful growth toward targets, with no analysis of long-term stability, regeneration, or behaviour at higher resolutions.

Neither project integrates with a 3D modelling environment, which means there is no practical workflow for preparing real mesh targets or observing training interactively.

4.4 Chapter Conclusion

The works reviewed in this chapter share a common foundation – local perception, shared update rules, residual dynamics and gradient-based optimisation – but none provide the tooling needed for systematic 3D experimentation. The original 2D Growing NCA established that a network with fewer than 10 000 parameters can learn to grow, maintain and regenerate complex patterns through purely local interactions. Subsequent 3D extensions confirmed that these principles transfer to volumetric grids, but left open questions about stability, persistence and regeneration that the experiments in this thesis aim to address.

Chapter 5

Architectural Design

This chapter presents the design of the 3D NCA framework and its overall architecture. It introduces the key components that form the framework and describes how they interact. The aim is to provide a clear conceptual overview of the system and explain the role of each part. Implementation-specific details, such as the libraries and frameworks chosen, are intentionally omitted here and discussed separately in the Implementation chapter.

5.1 Design Goals and Requirements

The purpose of the framework is to provide an environment for experimenting with neural cellular automata operating on three-dimensional voxel grids. To support experimentation and future extensions, the architecture requires a clear structure and well-defined interactions between components.

The primary design goal is to enable simulation and training of 3D neural cellular automata within a modular architecture. Components such as the simulation grid, update mechanism and training pipeline can be developed and modified independently, which allows different models and update rules to be tested without touching unrelated parts of the system.

A second goal was to keep the core functionality independent from visualisation tools and external applications. The framework is therefore designed as a standalone module that provides everything needed to initialise and train neural cellular automata. Other systems can build on this module and reuse its computational components without inheriting unnecessary dependencies.

During development, practical constraints influenced the design. Combining interactive visualisation with computationally intensive training within a single process proved difficult, as the two place conflicting demands on execution environments. This motivated a clean separation between computation and visualisation. The core framework provides the underlying simulation and training functionality, while additional components use it to offer visualisation and interaction. This approach preserves the independence of the core while allowing it to be integrated into different workflows.

5.2 System Overview

The framework is organised into three layers that together provide an interactive environment for training and visualising three-dimensional neural cellular automata. Figure 5.1 shows the high-level structure and the communication paths between these layers.

The innermost layer is the **core framework**. It contains the NCA model and the training pipeline and has no dependency on any server or graphical environment. It can be used directly from Python scripts or Jupyter notebooks, as the accompanying experiment notebooks demonstrate.

The second layer is the **server**, which exposes the core framework over a TCP connection. A running server accepts a single client connection, receives training configuration and target data, delegates execution to the core training pipeline and streams state snapshots back to the client as training progresses.

The third layer is the **Blender add-on**, which acts as the graphical interface and TCP client. It connects to the server, provides a panel-based interface for configuring and controlling training sessions and renders incoming state broadcasts as three-dimensional voxel meshes in the Blender viewport.

This separation serves two purposes. First, training can run on a remote GPU machine or a Jupyter notebook while the user interacts with Blender on a local workstation, with a tunnelling service bridging the two. Second, the core framework remains independent of both the server and the graphical environment, so researchers who do not need interactive visualisation can import and use it directly in their own scripts or notebooks.

The repository also contains lightweight notebook utilities for basic mesh-to-voxel conversion and offline visualisation. These were used during early exploration and cover only simple targets: they do not preserve mesh materials and do not perform per-voxel colour sampling. The Blender add-on replaces them for practical target preparation and interactive visualisation.

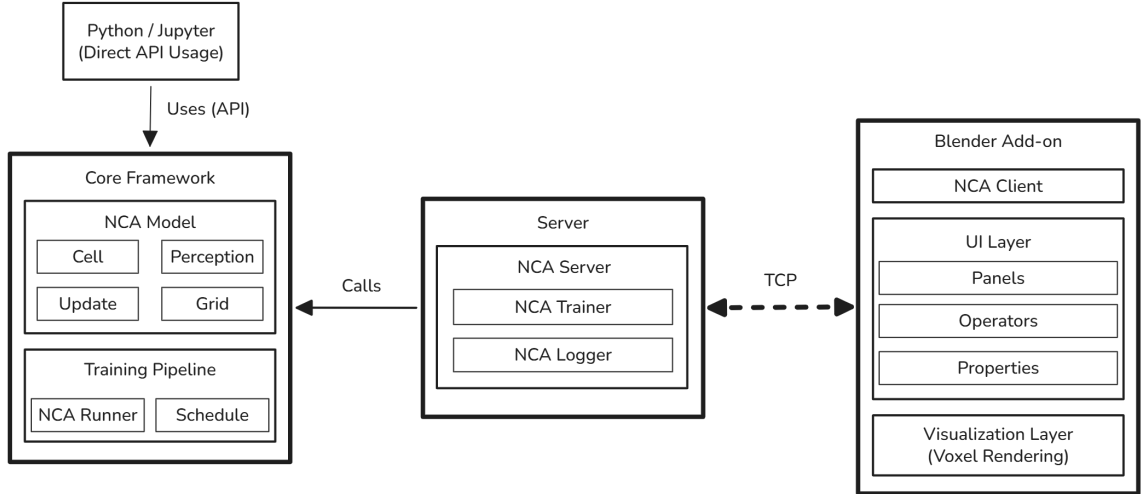


Figure 5.1: High-level system architecture. Arrows indicate dependency direction. The dashed TCP boundary marks the network separation between the server and the Blender client.

5.3 Core NCA Model

The core NCA model is composed of four different components: `CellState`, `Perception3D`, `UpdateRule` and `Grid3D`. Each component has its own configuration and can be instantiated independently. `Grid3D` acts as the composition root, wiring the other three together and exposing a single-step forward pass. A higher-level wrapper, `NCAModel`, bundles all configurations into one object and delegates to `Grid3D` internally, providing a simpler entry point for common use cases.

Keeping configurations and components separate means that experimenting with a custom update rule or a different perception kernel requires replacing only that component, leaving the rest of the pipeline unchanged. The experiment notebooks included with the thesis use the model components directly, without the runner or the server.

5.3.1 Cell

The cell component defines the structure of the per-voxel state vector and computes the alive mask that gates cell updates. It is configured by the number of hidden channels, the number of visible channels and the alive threshold.

The state vector of each voxel contains four groups of channels, as shown in Figure 5.2. Hidden channels carry internal memory that is not directly supervised and is not compared against the target during training. Visible channels represent the quantities compared against the target, such as RGBA colour values or a single occupancy channel. An optional third group, task channels, allows a single model to be trained on multiple targets simultaneously using a one-hot encoding to identify the active target. Task channels remain constant throughout training: the update rule explicitly zeroes their contribution to the state increment so they cannot be modified by the learned dynamics.

For asymmetric targets, an additional optional group of positional channels can be included. These channels store normalised absolute coordinates $(x, y, z) \in [0, 1]^3$ and serve as read-only constants, re-injected after every step to provide each cell with a stable global spatial reference. This allows cells to distinguish, for example, top from bottom or left from right, which is not possible from purely local neighbourhood statistics alone.

Task and positional channels are optional extensions that can be enabled per experiment. Baseline single-target training uses hidden and visible channels only.

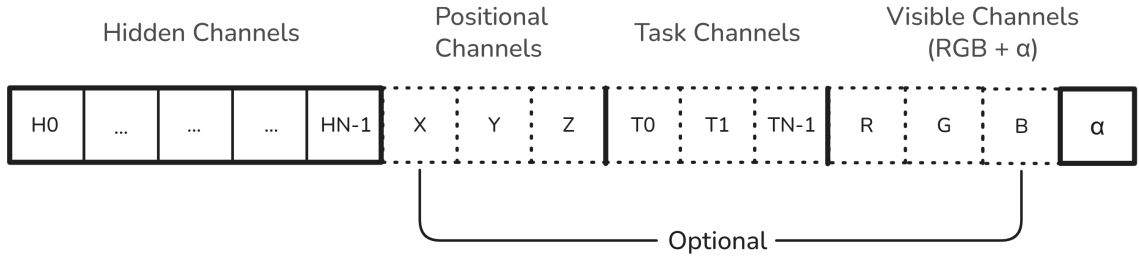


Figure 5.2: Per-voxel state vector layout. The vector contains hidden channels, optional positional channels, optional task channels and visible channels. The alpha channel, which serves as the alive signal, is always the last visible channel.

The alive mask controls which regions of the grid are active. It is computed by applying a $3 \times 3 \times 3$ max-pool over the alpha channel and comparing the result against the configured

threshold. A cell is considered potentially alive if it or any of its 26 neighbours carries an alpha value above the threshold. A dead cell adjacent to a live one therefore falls within the mask and can receive nonzero updates, which is what allows new cells to grow at the boundary of the developing structure [13].

5.3.2 Perception

The perception component converts the current grid state into a per-cell representation by applying a fixed set of $3 \times 3 \times 3$ convolutional filters over the local neighbourhood. The filters are non-learnable: their weights are set at initialisation and held constant throughout training.

In the baseline configuration, three filter groups are applied per channel: identity, six-neighbour sum and Laplacian. For asymmetric targets, a directional mode replaces the Laplacian with three signed gradients along the x , y and z axes, giving five groups in total. The signed gradients introduce orientation-sensitive features that break reflection ambiguity, which arises when a purely symmetric neighbourhood representation cannot distinguish a structure from its mirror image. Without these directional cues, the learned rule may converge to mirrored solutions that match occupancy statistics but not global orientation.

Following Mordvintsev et al. [13], keeping the filters fixed means the perception stage has no trainable parameters and cannot overfit independently. Translation invariance is built into the filter layout rather than learned from data. All trainable capacity resides in the update rule.

5.3.3 Update Rule

The update rule maps each cell’s perception vector to a state increment. Its configuration controls the capacity of the underlying network and whether stochastic update masking is active.

The increment is added residually to the current state:

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \Delta \mathbf{s}.$$

A bounded output activation limits the magnitude of each per-step update, preventing large state jumps when the model is unrolled over many steps and contributing to stable long-horizon dynamics [13].

The same update network is applied independently at every cell position, preserving the locality constraint. No cell has a privileged role by architecture; any functional differentiation must emerge from local context accumulated across steps.

When stochastic masking is enabled, each step randomly selects which cells apply their computed increment. This avoids lockstep synchronisation artefacts and makes the dynamics more tolerant of asynchronous perturbations, as discussed in the Theoretical Foundations chapter.

5.3.4 Grid and Model Composition

`Grid3D` composes the three components above into a module that executes one complete NCA update step. Figure 5.3 shows the data flow through a single step.

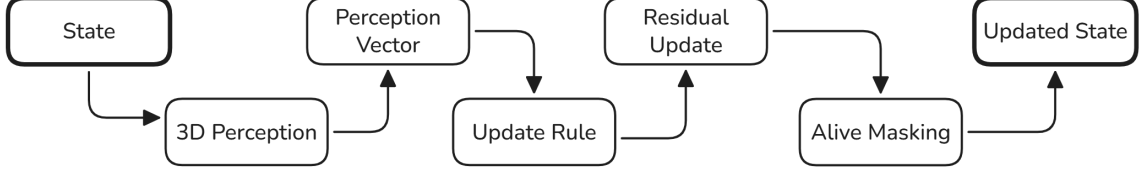


Figure 5.3: Data flow through a single NCA update step.

Each step passes the current state through the perception filters to produce a neighbourhood representation at every cell location. The update network maps these representations to bounded state increments, which are added residually. An alive mask is then computed from the updated state and applied to zero out any cell whose alpha falls below the threshold. This final masking step is what enables growth: dead cells adjacent to live ones receive nonzero increments and, if their resulting alpha crosses the threshold, become part of the active structure.

`NCAModel` is a higher-level entry point that bundles all component configurations and delegates to `Grid3D` internally. A saved checkpoint stores the full configuration alongside the learned weights, making it self-describing and loadable for inference without access to the original training setup.

5.4 Training Pipeline

The training pipeline sits above the core NCA model and provides the infrastructure needed to optimise the model parameters toward a target voxel configuration. It consists of two components: a training runner, which implements the training loop and a schedule manager, which handles dynamic parameter changes during training. The runner boundary is defined by an abstract `NCARunner` interface, so the server can accept a custom backend without any changes to the protocol or the Blender add-on.

Alongside the default implementation, the core package includes a regeneration-focused variant, `RegenRunner`, which applies random local damage to pooled states during training so the model can learn recovery behaviour through the same server and protocol path. More specialised variants, such as metamorphogenesis with explicit task-channel switching, are kept as notebook-level runners rather than fixed core classes.

5.4.1 Runner Interface and Default Implementation

The default `MorphRunner` implements pool-based training. Rather than restarting from a fresh seed at every iteration, it maintains a fixed-size pool of persistent states covering various stages of development. Each training iteration samples a batch from the pool, runs it forward for a randomly chosen number of steps, computes the loss against the target and writes the updated states back into the pool. The state with the highest loss is replaced by a fresh seed at each iteration, preserving diversity across developmental stages and preventing the pool from collapsing to a single end-state [13].

Any backend that satisfies the `NCARunner` interface serves as a drop-in replacement. For experimental work, this supports a notebook-first workflow: a custom runner can keep additional parameters hardcoded and treat the initialisation call primarily as a server handshake. Sending a minimal configuration from Blender remains valid for visualisation-only sessions, provided the custom runner emits snapshots in the format the server expects.

Task channels are a framework-level extension intended for notebook experiments and custom runners, not for the default Blender add-on configuration. When implementing such runners, the event handler should cover all relevant event types explicitly to avoid protocol defaults interfering with specialised training logic.

The training objective combines three weighted terms. Alpha loss compares the predicted and target occupancy channels across the full grid. Colour loss compares RGB values only where the target contains alive voxels, keeping the colour gradient signal focused on occupied regions. Overflow loss penalises alpha values in regions where the target is empty, limiting unwanted growth outside the target boundary. All three weights can be adjusted at runtime through the schedule system.

The number of forward steps per training iteration increases linearly over the first phase of training. Early iterations use short unrolls, keeping gradient paths manageable and allowing the model to learn coarse structure quickly. Later iterations use longer unrolls, testing whether the learned rule holds the target configuration stable over longer runs. Both the minimum and maximum step counts increase linearly over a fixed number of epochs and then stabilise at their final values.

5.4.2 Schedule

The schedule component manages one-shot events that fire at specified training epochs. Supported event types in the default runner include learning rate changes, batch size adjustments, individual loss weight updates and target replacements that swap the training objective mid-session. Events are thread-safe: the training loop and the server message handler share access through a lock. A special epoch value fires an event immediately on the next training step, allowing parameter adjustments during live training without restarting the session.

At execution time, the schedule delivers each due event to the runner. The event is one-shot and removed from the queue regardless of outcome. If a custom runner does not handle a given event type, the attempt is logged and the event is discarded.

5.5 Server and Networking

The server layer exposes the training pipeline over a TCP connection. It consists of three components: `NCAServer`, which handles client connections and dispatches messages; `NCATrainer`, which executes training and inference in a background thread; and `NCALogger`, which persists run metadata and artefacts to disk. A shared protocol module defines the message format used across the boundary between the server and the Blender client.

5.5.1 Connection Architecture

The server binds to a configurable host and port and accepts one client at a time. For each connection, it enters a message dispatch loop that routes incoming messages to the

appropriate handler and sends responses back. When the client disconnects or sends a **stop** message, any active session is terminated and the server returns to listening.

NCATrainer runs the training or inference loop in a background thread, keeping protocol handling independent of the computationally intensive session work. This separation maintains clear session lifecycle boundaries and keeps the dispatch loop responsive regardless of training load.

Figure 5.4 shows the full sequence of messages exchanged during a typical session, from connection through training, schedule update, pause and resume, to inference and final disconnect.

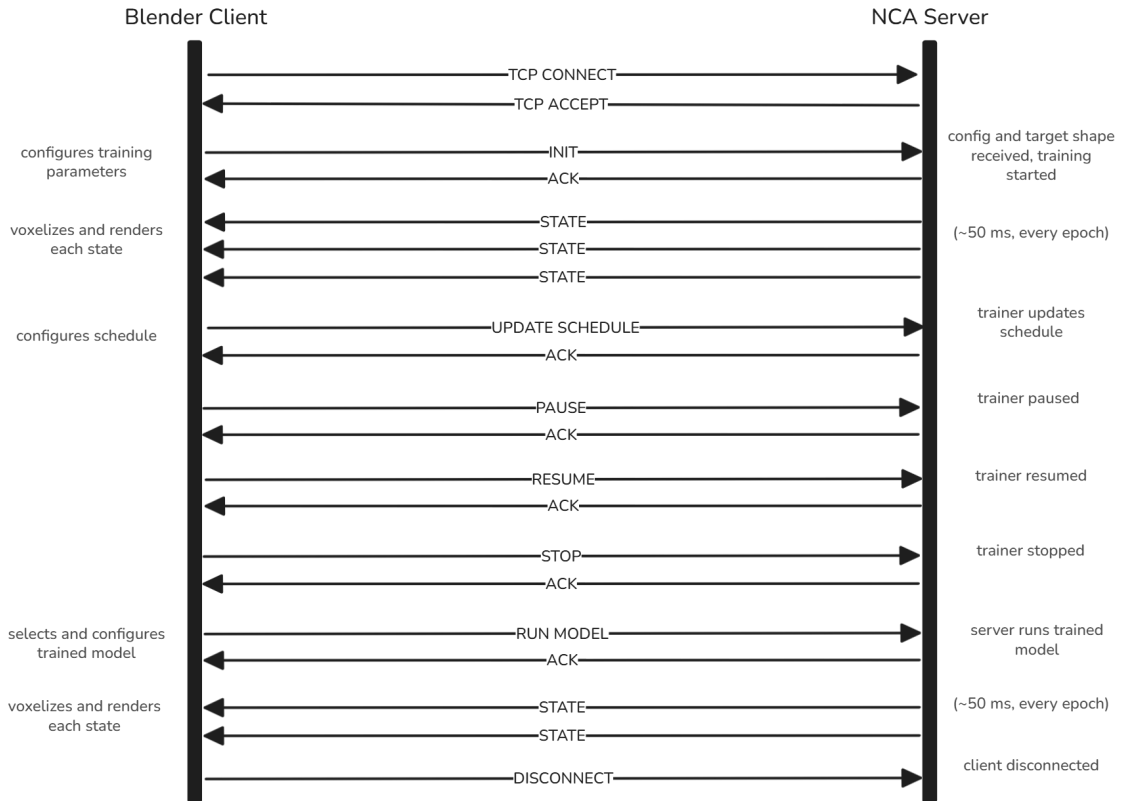


Figure 5.4: Client-server message sequence for a complete session. The client drives all state transitions; the server responds with acknowledgements and streams state broadcasts during active training or inference.

Figure 5.5 shows the session state machine from the server perspective. A session begins when the client sends an **init** message for training or a **run_model** message for inference and ends when the client sends **stop** or disconnects. Training sessions support **pause** and **resume** transitions. Inference sessions share the same pause mechanism at the implementation level, but the Blender add-on does not expose pause controls during inference, so in practice inference runs to completion or until stopped.

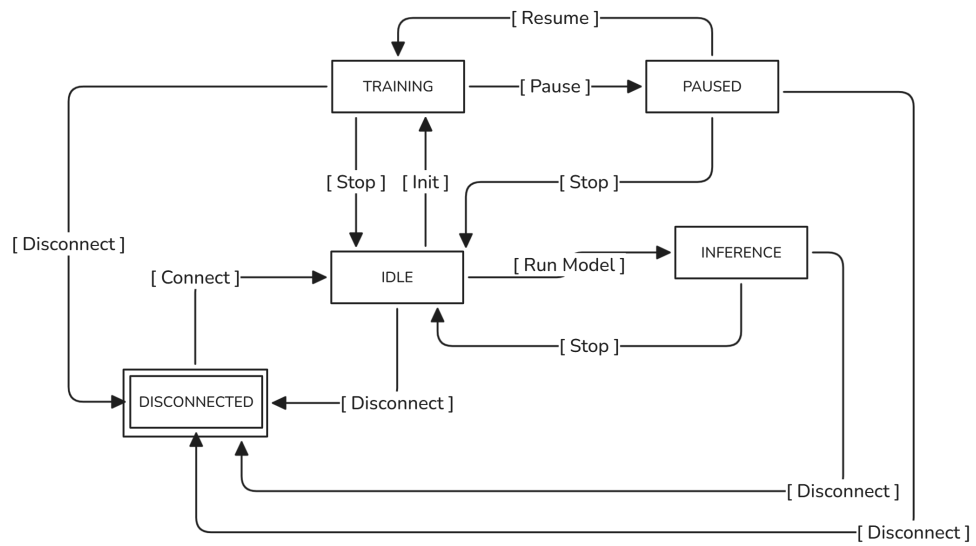


Figure 5.5: Session state machine. The server transitions between disconnected, idle, training, paused and inference states in response to client messages. All active states return to disconnected if the client closes the connection.

5.5.2 Protocol

TCP provides an ordered, reliable byte stream but does not preserve message boundaries [5]. The protocol therefore uses explicit framing: each message consists of a fixed-size length header followed by a structured text payload. Binary data, such as state snapshots and model checkpoints, are encoded as byte strings and carried as fields within the payload. This keeps a single message format across all message types and avoids a separate binary framing layer.

The protocol supports two directions. Clients send control messages: `init` (start training), `run_model` (start inference), `stop`, `pause`, `resume` and `update_schedule`. The server responds with `ack` confirmations or `error` messages. During an active session, the server additionally streams `state` messages containing the current grid snapshot, epoch number and loss value. The client drives all state transitions; the server streams state and responds to control requests.

5.5.3 Logging

The logger saves training metadata and results to a structured directory layout. Each training run occupies its own directory with an automatically assigned run ID, so concurrent or sequential runs never collide. Within a run directory, a metadata file records the full configuration and start timestamp, a loss log records per-epoch metrics and an event log records fired schedule events. Model checkpoints are written at a configurable interval and a final checkpoint is always saved when training completes, regardless of interval setting.

Each checkpoint stores the model configuration alongside the learned weights, making it self-describing: it can be loaded for inference without access to the original training setup.

5.6 Blender Integration

The Blender add-on is the primary graphical interface for the framework. It covers three concerns: mesh voxelization for target preparation, configuration and control panels and viewport visualisation of training progress and inference.

5.6.1 Voxelization

The add-on converts Blender mesh objects into voxel arrays that serve as training targets. A mesh selected in the viewport is sampled onto a three-dimensional grid whose dimensions are set by the user. The voxelized result is displayed in the viewport alongside the original mesh so the user can inspect the discretization before starting training.

Colour information is extracted from mesh materials during voxelization. Two modes are available. In BSDF mode, each occupied voxel takes the base colour directly from the nearest surface material. In texture mode, the colour is sampled from an image texture at the nearest surface point, which handles imported meshes with image-based materials that BSDF mode cannot read. The mode is configurable per session.

Voxelized targets can be exported to disk for use in external training scripts or notebooks, giving users access to the voxelization pipeline without requiring the full server workflow.

5.6.2 User Interface

The add-on provides a hierarchical panel layout in the Blender sidebar, organised under a single root panel. The panels cover connection settings, training control, target configuration, inference parameters, model component settings and schedule management.

Connection state and session state determine which controls are active. Configuration panels are disabled during an active session so settings that cannot be changed mid-training remain read-only. Starting training requires a voxelized target and a connected server. Starting inference requires a connected server and a path to a saved model checkpoint. The two modes are mutually exclusive and both return the server to idle when stopped.

5.6.3 Visualisation

State broadcasts received from the server are rendered as voxel meshes in the Blender viewport. The add-on maintains three spatial slots arranged side by side: the original source mesh, the voxelized target and the current NCA state received from the server. Placing all three in the same scene lets the user compare the source, the discretized target and the evolving model output at the same time.

The state mesh is rebuilt from each received broadcast. A voxel cap prevents display from overwhelming the GPU on dense grids; when the active cell count exceeds this limit, only the highest-alpha voxels are retained.

Received state tensors are converted to the display format in a dedicated adapter step before rendering. This keeps the visualisation code independent of internal tensor layouts, so alternative state formats can be supported by updating the conversion logic rather than the rendering or protocol code.

The network listener and the Blender main thread run separately. The listener passes incoming states through a shared buffer and the main thread drains that buffer and performs all viewport updates. This ensures scene data is only ever modified from the thread that owns it.

Chapter 6

Implementation

This chapter describes how the framework is implemented in code. It covers the project structure, the main libraries and the NCA model components shared by training scripts, the TCP server and the Blender add-on.

6.1 Core Framework

The implementation is organised into the following Python packages:

- `nca3d/core/` – NCA model, stepping logic and training utilities,
- `nca3d/server/` – TCP server, protocol handling and the training runner,
- `nca3d/io/` – legacy import/export and mesh conversion utilities used in early notebooks,
- `nca3d/viz/` – notebook visualisation helpers for offline inspection,
- `nca3d/blender/` – Blender add-on for voxelisation, UI and viewport rendering.

Voxel targets for training are prepared and exported through the add-on, which samples colour from Blender materials and provides a single place to adjust grid size and training presets. The notebook utilities in `nca3d/io/` and `nca3d/viz/` cover only simple cases and are not part of the interactive workflow.

PyTorch provides tensor operations, 3D convolutions and automatic differentiation for unrolling the automaton through time [17, 3]. NumPy is used at I/O boundaries and for data structures that need to be serialised or passed across the client-server boundary.

The simulator stores the full grid state as a single float32 tensor of shape `[B, C, D, H, W]`, where `B` is batch size, `C` is the total number of channels and `D, H, W` are the grid dimensions. This layout matches the convention expected by 3D convolution operations and avoids redundant transpositions inside the perception and update steps. Listing 6.1 shows the two channel slices used most often across the codebase.

```
1 # state: [B, C, D, H, W]
2 # V: number of visible channels (e.g. 4 for RGBA)
3 visible = state[:, -V:, ...]
4 alpha = state[:, -1:, ...]
```

Listing 6.1: Common channel slices into the state tensor.

Channel ordering follows the state vector layout shown in Figure 5.2. Hidden channels come first, followed by optional positional channels, optional task channels and visible channels, with alpha stored last. Task and positional channels are disabled in the baseline configuration and can be enabled per experiment.

Model hyperparameters are grouped into small configuration dataclasses. The four component-level configs – `CellConfig`, `PerceptionConfig`, `UpdateConfig` and `GridConfig` – are frozen, so they can be passed around safely and used as serialisation inputs for logs and checkpoints. `NCAConfig` collects the most commonly adjusted parameters into a single object for use with the high-level `NCAModel` wrapper.

The client-server setup exchanges configuration and voxel targets across the network. The concrete message structure and the conversion between external voxel arrays and internal tensors are described in Sections 6.3 and 6.4.

6.2 NCA Model Components

The architectural roles of the NCA components are described in Section 5.3. This section focuses on how that design maps to Python modules, configuration objects and entry points.

The NCA model follows the standard two-stage structure: a fixed perception operator extracts local features and a learned update rule predicts a per-voxel state increment. The implementation is split into four modules:

CellState channel layout and alive-mask computation,

Perception3D fixed depthwise `Conv3d` for local perception,

UpdateRule point-wise MLP implemented with $1 \times 1 \times 1$ convolutions,

Grid3D one-step update logic and multi-step unrolling.

`NCAModel` wraps `Grid3D` and provides a single entry point with standardised checkpointing.

The server and runner build `Grid3D` directly from individual component configs, which gives full control when custom behaviour or modified components are needed. Notebooks and scripts can use `NCAModel` instead, either with a single `NCAConfig` object or through keyword arguments that mirror the same fields. Listings 6.2 and 6.3 show both patterns.

```

1 cell_cfg = CellConfig(
2     hidden_channels=16,
3     visible_channels=4,
4     alive_threshold=0.1,
5     task_channels=0,
6     pos_channels=3,
7 )
8 perc_cfg = PerceptionConfig(kernel_radius=1, channel_groups=5)
9 upd_cfg = UpdateConfig(hidden_dim=128, stochastic_update=False,
10                        fire_rate=0.5)
11 grid_cfg = GridConfig(size=(32, 32, 32))
12 grid = Grid3D(cell_cfg, perc_cfg, upd_cfg, grid_cfg)

```

Listing 6.2: Low-level `Grid3D` construction from individual component configs, used when custom behaviour or modified components are needed.

```

1 model = NCAModel(
2     grid_size=(32, 32, 32),
3     hidden_channels=16,
4     visible_channels=4,
5     alive_threshold=0.1,
6 )

```

Listing 6.3: High-level NCAModel wrapper for standard initialisation via keyword arguments.

6.2.1 CellState

The per-voxel state vector layout follows Figure 5.2. `CellConfig` specifies the channel counts and `CellState.update_alive_mask()` computes the boolean alive mask used to gate updates. A voxel is considered alive if the maximum alpha value in its $3 \times 3 \times 3$ neighbourhood exceeds `alive_threshold`. Listing 6.4 shows the core computation.

```

1 alpha = state[:, -1:, ...]
2 pooled = F.max_pool3d(alpha, kernel_size=3, stride=1, padding=1)
3 alive = pooled > alive_threshold

```

Listing 6.4: Alive-mask computation from the alpha channel.

6.2.2 Perception3D

`Perception3D` implements the fixed perception stage as a depthwise `Conv3d`. In the baseline configuration, three filter groups per input channel produce 3C output features. In the directional configuration, five groups produce 5C features by replacing the Laplacian with signed gradients along each axis. Weights are initialised once and remain fixed during training. Listing 6.5 shows the kernel patterns in conceptual form.

```

1 # identity
2 K_id[1, 1, 1] = 1
3
4 # 6-neighbour sum (face-adjacent voxels)
5 for (dz, dy, dx) in [(1,0,0),(-1,0,0),(0,1,0),(0,-1,0),(0,0,1),(0,0,-1)]:
6     K_sum[1+dz, 1+dy, 1+dx] = 1
7
8 # optional signed directional gradients
9 K_dx[2, 1, 1] = 1; K_dx[0, 1, 1] = -1
10 K_dy[1, 2, 1] = 1; K_dy[1, 0, 1] = -1
11 K_dz[1, 1, 2] = 1; K_dz[1, 1, 0] = -1

```

Listing 6.5: Perception kernels applied per input channel (conceptual form).

6.2.3 UpdateRule

`UpdateRule` maps perception features to a per-voxel state increment. It is a point-wise MLP using $1 \times 1 \times 1$ convolutions, so all spatial mixing is handled by the perception stage. The final layer is initialised to zero, which keeps the initial dynamics close to an identity update and avoids large early perturbations.

The raw output is passed through `tanh` and scaled by 0.1, bounding each per-step increment. An optional stochastic fire mask applies the update only to a randomly selected subset of voxels per step. Listing 6.6 shows the full layer sequence.

```

1 mlp = nn.Sequential(
2     nn.Conv3d(G*C, H, 1),
3     nn.GroupNorm(4, H),
4     nn.ReLU(inplace=True),
5     nn.Conv3d(H, H, 1),
6     nn.GroupNorm(4, H),
7     nn.ReLU(inplace=True),
8     nn.Conv3d(H, C, 1), # zero-initialised
9 )
10 dx = 0.1 * torch.tanh(mlp(perceived))

```

Listing 6.6: Update rule MLP implemented with $1 \times 1 \times 1$ convolutions.

When task or positional channels are present, the predicted increment for those slices is forced to zero, keeping them read-only with respect to the learned update.

6.2.4 Grid3D

Grid3D composes `CellState`, `Perception3D` and `UpdateRule` into a complete simulator. Each step applies a residual update and then masks out voxels that fall below the alive threshold. Listing 6.7 summarises the logic.

```

1 pre_life = cell.update_alive_mask(state)
2 perceived = perception_op(state)
3 dx = update_rule(perceived, pre_life, state)
4 new_state = state + dx
5
6 post_life = cell.update_alive_mask(new_state)
7 new_state = new_state * post_life.float()
8
9 # positional channels re-injected as read-only constants
10 new_state[:, pos_slice] = pos_grid

```

Listing 6.7: Single NCA update step (pseudocode).

The multi-step `forward()` loop calls `step()` repeatedly and clamps the state to $[-1, 1]$ after each update. For long training unrolls, gradient checkpointing can be enabled to reduce peak memory usage at the cost of recomputation during backpropagation.

Grid3D also provides `seed_center()`, which creates an initial state with a single active voxel at the lattice centre. Visible channels are initialised to random colour values with alpha set to 1.0 and hidden channels start at zero. When positional channels are enabled, they are filled with normalised coordinate values across the full grid and re-injected after each step.

6.2.5 NCAModel

NCAModel wraps Grid3D and keeps all configuration in a single object for use in scripts and notebooks. Checkpoints store a configuration snapshot alongside the learned weights, making them self-describing. Listing 6.8 shows the structure written by NCAModel.save().

```

1 torch.save(
2     {
3         "version": 1,
4         "config": dataclasses.asdict(model.config),
5         "state_dict": model.state_dict(),
6     },
7     path,
8 )

```

Listing 6.8: Checkpoint structure written by NCAModel.save().

Figure 6.1 shows the full layer sequence through a single update step. The diagram reflects the default three-group perception configuration; the directional five-group variant follows the same structure.

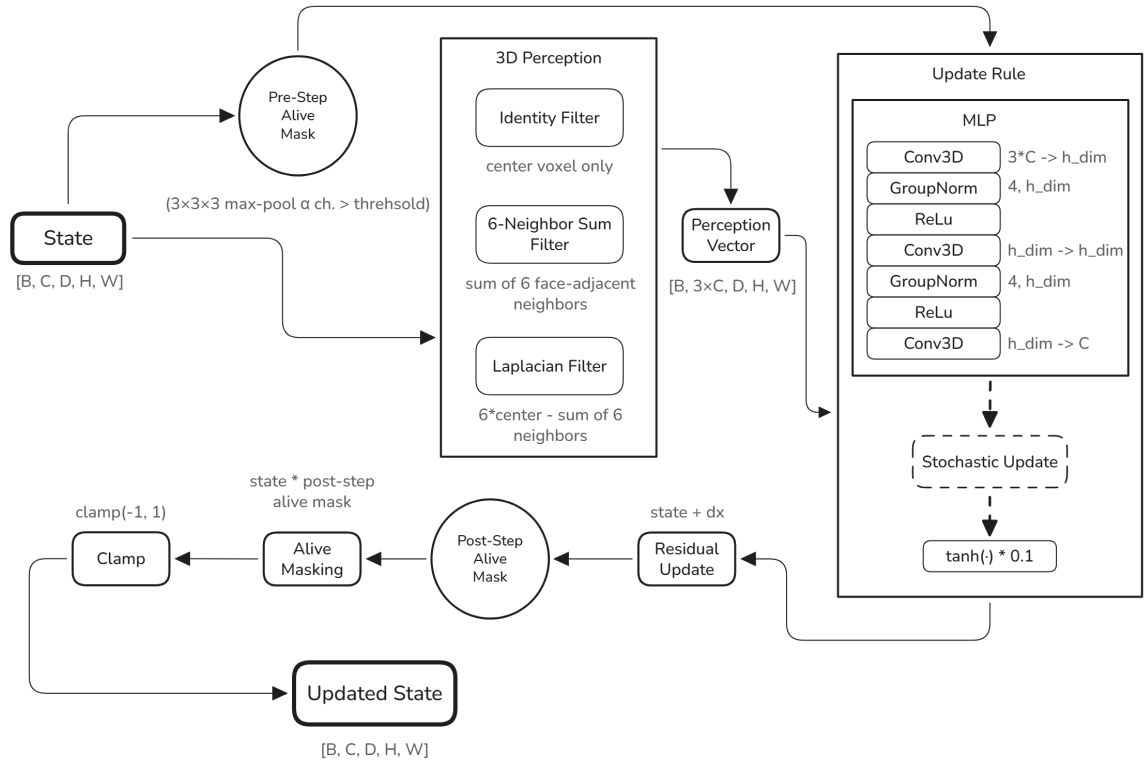


Figure 6.1: Detailed data flow through a single NCA update step, showing the full layer sequence within the update MLP, the optional stochastic fire mask and the bounded output. The diagram uses the default three-group perception configuration; the directional five-group variant follows the same structure.

6.3 Training Pipeline

The conceptual structure of pool-based training, curriculum stepping and schedule-driven parameter updates is described in Section 5.4. This section focuses on the code-level composition and the data formats used at the training boundary.

Training backends are defined by the abstract `NCARunner` interface. The interface is defined in `nca3d/core/runners/base.py`. The default implementation, `MorphRunner`, owns the model instance (`Grid3D`), the optimiser, the current target tensor and a pool of intermediate states used as training samples. The main entry point is the generator method `train()`, which yields per-epoch metrics and allows the server to stream progress without blocking on GPU work.

The second core backend, `RegenRunner` (`nca3d/core/runners/regeneration.py`), uses the same optimisation path but injects random local box damage into pooled states before each update, enabling regeneration experiments without changes to the server or protocol. Damage probability is controlled by `training.regen_damage_prob`. Damage size is controlled by `training.regen_damage_size`.

Runner initialisation accepts voxel targets in the channels-last format `[D, H, W, C]`, which is convenient for Blender and for protocol messages. Before training begins, the array is converted to a channels-first tensor `[B, C, D, H, W]`. Listing 6.9 shows the conversion.

```
1 # NumPy [D, H, W, C] -> torch [B, C, D, H, W]
2 x = (torch.from_numpy(np_array)
3      .permute(3, 0, 1, 2)
4      .unsqueeze(0)
5      .contiguous())
```

Listing 6.9: Conversion from channels-last voxel array to channels-first training tensor.

Pool-based training draws samples from a pool of previously generated states rather than always restarting from a clean seed, which exposes the model to different partial solutions and stabilises long-horizon behaviour. The step count per epoch is drawn randomly from a range that widens over training: early epochs use short unrolls to learn coarse structure, while later epochs use longer ones. Both mechanisms are covered in Section 5.4.

Visible channels are supervised directly against the target; hidden channels remain unsupervised. The total loss is a weighted sum of three terms: an alpha occupancy term, an RGB colour term (when colour channels are present) and an overflow penalty that discourages activity outside the intended volume. Listing 6.10 summarises one training epoch.

```
1 batch = sample(pool, batch_size)
2 steps = randint(step_min(epoch), step_max(epoch))
3
4 pred = model(batch, steps=steps)
5 loss = (
6     w_alpha * loss_alpha(pred, target)
7     + w_color * loss_color(pred, target)
8     + w_overflow * loss_overflow(pred)
9 )
10 loss.backward()
11 optimizer.step()
12 pool.update(batch, pred)
```

Listing 6.10: One training epoch in the pool-based runner (pseudocode).

Task channels are available at the model level but are not used by either core runner. The notebook-level metamorphogenesis experiment demonstrates them via a custom runner that manipulates channel slices before broadcasting snapshots. Schedule events – learning rate, batch size, loss weights and target changes – are applied through the thread-safe `Schedule` class.

6.4 Server

The server architecture and session lifecycle are described in Section 5.5, including the message sequence (Figure 5.4) and the protocol state machine (Figure 5.5). This section covers the code structure of the TCP server and the background training thread.

Training sessions start with an `init` message that carries a configuration dictionary and the voxel target. Listing 6.11 shows the top-level structure exchanged between Blender, the server and the core runner.

```

1 config = {
2     "cell": { ... },
3     "perception": { ... },
4     "update": { ... },
5     "grid": { ... },
6     "training": { ... },
7 }
```

Listing 6.11: Top-level configuration structure exchanged between Blender and the training server.

On the server side, each sub-dictionary is unpacked directly into the corresponding configuration object. Listing 6.12 shows the mapping.

```

1 cell_cfg = CellConfig(**config["cell"])
2 perc_cfg = PerceptionConfig(**config["perception"])
3 upd_cfg = UpdateConfig(**config["update"])
4 grid_cfg = GridConfig(**config["grid"])
```

Listing 6.12: Mapping a configuration dictionary to component configuration objects.

`NCAServer` handles the TCP socket lifecycle and dispatches incoming messages. Computationally intensive work is delegated to `NCATrainer`, which runs in a background thread and exposes a control API: `pause`, `resume`, `stop` and `update_schedule`. During training, the trainer drives the selected `NCARunner.train()` generator and broadcasts state snapshots to the client with rate limiting, keeping viewport updates smooth when the training loop runs faster than the network link. During inference, the trainer loads a checkpoint from raw bytes and runs forward steps without gradient checkpointing.

Listing 6.13 shows the message dispatch loop in simplified form.

```

1 while True:
2     msg = recv_msg(sock)
3     t = msg.get("type")
4
5     if t == "init":
6         trainer.init(config, target, send_fn)
7     elif t == "run_model":
8         trainer.run_inference(
9             model_bytes, phase_steps, broadcast_every, send_fn)
10    elif t == "pause": trainer.pause()
```

```

11 elif t == "resume": trainer.resume()
12 elif t == "stop": trainer.stop()
13 elif t == "update_schedule":
14     trainer.update_schedule(events)

```

Listing 6.13: Server-side message dispatch loop (simplified).

Listing 6.14 summarises the typical call patterns for the trainer API.

```

1 trainer = NCATrainer(base_dir="runs", checkpoint_interval=500)
2
3 trainer.init(config, target, send_fn) # start a training run
4 trainer.update_schedule(events)
5 trainer.pause()
6 trainer.resume()
7 trainer.stop()
8
9 # inference from a raw checkpoint sent by the client
10 trainer.run_inference(
11     model_bytes, phase_steps, broadcast_every, send_fn)
12
13 # inject a custom backend
14 trainer = NCATrainer(runner_factory=MyCustomRunner)

```

Listing 6.14: Typical trainer usage patterns from the server layer (pseudocode).

For notebook or Colab experiments with Blender visualisation, a small `NCARunner` subclass can be defined in a notebook cell and injected via `runner_factory`. Listing 6.15 shows this pattern.

```

1 class NotebookRunner(NCARunner):
2     def __init__(self):
3         self.base = MorphRunner(verbose=False)
4     def init(self, config, target):
5         self.base.init(config, target)
6     def train(self, schedule=None):
7         yield from self.base.train(schedule=schedule)
8     def snapshot(self): return self.base.snapshot()
9     def set_target(self, t): return self.base.set_target(t)
10    def on_event(self, ev): return self.base.on_event(ev)
11
12 trainer = NCATrainer(runner_factory=NotebookRunner, verbose=False)
13 NCAServer(host="0.0.0.0", port=5555, trainer=trainer).start()

```

Listing 6.15: Custom runner injection for notebook experiments (simplified).

6.4.1 Protocol

Protocol behaviour at the session level is described in Section 5.5. This subsection covers the concrete framing and payload encoding used in the implementation.

Each message consists of a four-byte big-endian unsigned integer carrying the JSON payload length, followed by the UTF-8 JSON body (Table 6.1). Tensor payloads – targets and broadcast states – are transmitted as base64-encoded float32 byte arrays with an explicit `shape` field. Model checkpoints are transmitted as base64-encoded raw `.pt` bytes during inference requests. Listing 6.16 shows the framing operation used by both client and server.

```

1 payload = json.dumps(msg).encode("utf-8")
2 wire = struct.pack(">I", len(payload)) + payload
3 sock.sendall(wire)

```

Listing 6.16: Length-prefixed framing for TCP messages.

The `init` message is the primary extension point for custom experiments: fields can be added or reshaped to support different configuration structures or multi-target voxelisation. Control messages (`pause`, `resume`, `stop`) are intentionally stable, as they represent session-level transport control rather than experiment-specific logic. Table 6.2 summarises all message types.

Field	Value
Header	4 bytes; <code>uint32</code> (big-endian); N (payload length)
Payload	N bytes; UTF-8 JSON object (<code>type</code> + message fields)

Table 6.1: Wire message framing.

Message type	JSON fields
<i>Client → Server</i>	
<code>init</code>	<code>type</code> , <code>config{}</code> , <code>target</code> (base64), <code>target_shape[]</code>
<code>run_model</code>	<code>type</code> , <code>model_b64</code> , <code>phase_steps</code> , <code>broadcast_every</code> , <code>send_delay_ms</code>
<code>update_schedule</code>	<code>type</code> , <code>events[]</code>
<code>stop</code>	<code>type</code>
<code>pause</code>	<code>type</code>
<code>resume</code>	<code>type</code>
<i>Server → Client</i>	
<code>state</code>	<code>type</code> , <code>data</code> (base64), <code>shape[]</code> , <code>epoch</code> , <code>loss</code>
<code>ack</code>	<code>type</code> , <code>message</code>
<code>error</code>	<code>type</code> , <code>message</code>

Table 6.2: Protocol message types and their JSON fields.

6.4.2 Logging

The role of logging and stored artefacts is described in Section 5.5. In the implementation, run outputs are written by `NCALogger` (`nca3d/server/logger.py`). Each training run claims the next available `runs/run_NNN/` directory and writes the following files:

- `meta.json` – configuration snapshot, timestamp and optional git hash,
- `loss.csv` – per-epoch loss components,
- `events.jsonl` – applied schedule events, one JSON record per line,
- `checkpoints/` – periodic `.pt` checkpoints, each containing `config` and `state_dict`.

Listing 6.17 shows the on-disk layout produced for each run.

```

runs/
  run_003/
    meta.json

```

```

loss.csv
events.jsonl
checkpoints/
    model_ep00500.pt
    model_ep01000.pt
    ...

```

Listing 6.17: Run directory layout produced by the training server.

6.5 Blender Add-on

The add-on architecture is described in Section 5.6. This section covers the implementation structure and the threading pattern required for interactive visualisation.

The add-on is implemented in the self-contained `blender/` package following the standard Blender add-on structure. UI properties are stored in scene-level property groups and exposed through sidebar panels. Actions are implemented as Blender operators that connect to the server and request training or inference. Voxelisation and viewport rendering are handled by `voxel_utils.py`.

Network communication runs off the Blender main thread. `NCAClient` runs a listener thread that receives protocol messages and decodes broadcast states. The listener overwrites a shared buffer with the latest received state and schedules a timer callback if one is not already registered. The timer runs on the main thread and performs all scene updates, which is required because Blender does not allow scene modifications from background threads. Listing 6.18 shows the pattern in simplified form.

```

1 def on_state(arr):
2     global pending
3     pending = arr
4     bpy.app.timers.register(poll_state, first_interval=0.05)
5
6 def poll_state():
7     global pending
8     if pending is not None:
9         render_state(pending)
10        pending = None
11    return 0.1

```

Listing 6.18: State visualisation via a listener thread and a main-thread timer (simplified).

Chapter 7

Testing

The framework was tested at three levels. The automated suite targets specific code-level behaviours in the core model, runners and server. Protocol verification confirmed the wire format under real network conditions. And the Blender add-on, once implemented, became the main environment for observing training dynamics end to end. Most of the practical fixes and adjustments made during development came from that last category rather than from the automated tests alone. The experiments described in the following chapter extend this further and serve as functional validation of the framework as a whole.

7.1 Automated Tests

Automated tests are implemented using `pytest` in the `tests/` directory across five files.

`test_core.py` covers the core model components. It verifies the alive-mask computation in `CellState`, the fixed perception kernels and frozen weights in `Perception3D`, basic properties of `UpdateRule` including the zero-initialised final layer and masking of task-channel and positional-channel increments, and seed initialisation in `Grid3D` including the one-hot task-channel slice and positional channel re-injection after each step. Configuration and checkpoint round-trips for `NCAModel` are also covered.

`test_runner_validation.py` targets `MorphRunner` and `RegenRunner`. The tests cover initialisation, one-epoch training metrics, snapshot shape and content, event handling through `on_event()`, abstract-interface compliance and input-shape validation. It also checks that `MorphRunner` correctly rejects configurations with task channels enabled.

`test_protocol_schedule.py` contains runner-facing contract checks that partially overlap with `test_runner_validation.py`. It was retained as a lightweight regression file after the runner abstraction changes, covering snapshot shape and event handling at a slightly higher level than the runner tests.

`test_schedule_custom_events.py` tests the schedule system and how it handles custom event types in string format. It checks that built-in event type strings are normalised to `EventType` enums, that unknown types such as `POOL_EXPAND` are preserved as raw strings, and that `Schedule.check_and_execute` correctly delivers and removes such events.

`test_server_client.py` covers the server-client boundary. The Blender client is tested with a fake socket and a monkeypatched send function to verify that the correct control messages are produced. The server dispatch loop is tested against a fake trainer to confirm message routing, and broadcast rate limiting is verified separately. Two further tests cover checkpoint parsing for both the nested runner format and the flat `NCAModel` format.

7.2 Protocol Verification

Alongside the automated tests, Wireshark was used during interactive sessions to inspect raw TCP traffic between the Blender client and the server. This confirmed that message framing was correct and that broadcast timing matched the expected behaviour. It was most useful during early server development when the framing logic and rate limiting were still being adjusted.

7.3 Notebook Experiments

Before the Blender add-on was available, Jupyter notebooks served as an exploratory environment for the core model and training behaviour. These rely on the `nca3d/viz/` and `nca3d/io/` helpers, which is the main reason those modules exist despite not being part of the core-server-Blender workflow. Unlike the pool-based training used by the server runners, these notebooks use a simpler per-iteration loop that restarts from a fresh seed on each step. The four notebooks each target a different aspect of the system:

- `01_basic_functionality.ipynb` – stepping an untrained model forward and inspecting state statistics and alive voxel counts,
- `02_sphere_training_alpha.ipynb` – training on an analytic sphere target with alpha-only optimisation and slice and volume visualisations,
- `03_donut_training_alpha.ipynb` – training on a mesh-based target imported from an OBJ file in an occupancy-only setting,
- `04_donut_training_rgba.ipynb` – training on an RGBA donut target with colour-supervised loss applied to a cropped sub-region of the grid.

7.4 End-to-End Testing

After the Blender add-on was implemented it became the primary environment for end-to-end testing. Target voxelisation, protocol behaviour, training dynamics and viewport rendering were all observed interactively. Later Colab and Kaggle notebooks extended this to longer training runs, remote GPU workflows via ngrok tunnelling and multi-target metamorphogenesis with task channels. These experiments are discussed in the following chapter.

Chapter 8

Experiments

This chapter presents the experimental evaluation of the framework. The experiments were designed to answer several practical questions:

- Can the baseline 3D NCA grow a target from a single seed in a stable way?
- Can one model learn multiple targets and transition between them through task-channel conditioning?
- Does the interactive remote training workflow – core framework on a GPU machine, real-time visualisation in Blender over a TCP tunnel – work in practice?
- Can the framework be used effectively without the server, treating the core package as a standalone research tool?
- Does schedule-driven training remove the need to hardcode phase transitions in the training loop?

The focus is on morphogenesis as the primary training regime and metamorphogenesis as the more demanding multi-target extension. Regeneration is implemented and available through **RegenRunner** but no full qualitative experiment was completed within the scope of this thesis; it is left as future work. This reflects a deliberate choice to explore the breadth of the framework rather than exhaustively cover every training mode.

Unless stated otherwise, all experiments use a 32^3 grid and start from a single alive seed at the grid centre.

8.1 Morphogenesis

Morphogenesis experiments focus on single-target growth from a seed. They use the default **MorphRunner** training path and serve as a baseline for the later metamorphogenesis experiments.

Table 8.1 summarises the shared training settings implemented by **MorphRunner**. Unless stated otherwise, these apply to all morphogenesis experiments in this section.

Category	Setting	Value
Optimisation	Optimiser	AdamW (<code>weight_decay</code> = 10^{-5})
Optimisation	LR schedule	cosine annealing ($\eta_{\min} = 10^{-5}$)
Training dynamics	Pool size	32
Training dynamics	Step range	(8, 16) $\rightarrow$ (32, 64) over first 2000 epochs
Loss	Loss weights	$w_\alpha : w_c : w_o = 4:1:2$
Stability	Gradient clip	global norm 1.0

Table 8.1: Default **MorphRunner** training settings.

8.1.1 Symmetrical Morphogenesis

The first full morphogenesis experiment used a symmetric target with the configuration in Table 8.2.

Setting	Value
Grid size	32^3
<code>cell.hidden_channels</code>	16
<code>cell.visible_channels</code>	4 (RGBA)
<code>cell.task_channels</code>	0
<code>cell.pos_channels</code>	0
<code>perception.kernel_radius</code>	1
<code>perception.channel_groups</code>	3
<code>update.hidden_dim</code>	96
<code>training.learning_rate</code>	2×10^{-3}
<code>training.num_epochs</code>	3000
<code>training.batch_size</code>	4

Table 8.2: Symmetric morphogenesis configuration.

All other training settings follow Table 8.1.

This experiment was run in two configurations to validate two different framework workflows. The first used the full interactive pipeline: training ran on a remote Colab GPU and the Blender add-on connected to it over an ngrok TCP tunnel, rendering incoming state broadcasts in real time. This confirmed that the full three-layer pipeline (core, server, Blender) works end to end, including the tunnelling scenario described in the System Overview. The second used only the core package and **MorphRunner** in a Colab notebook without a server session, with Blender used separately for target preparation and post-training inference inspection. This is the connectionless researcher workflow: the server and add-on are optional, and the core package functions as a standalone training library.

Figure 8.1 shows the supervised target for RGBA reconstruction. The alpha channel acts as the alive signal and the RGB loss is masked by the target alpha, so only voxels above the alive threshold contribute to colour error.

Target (2248 surface voxels)

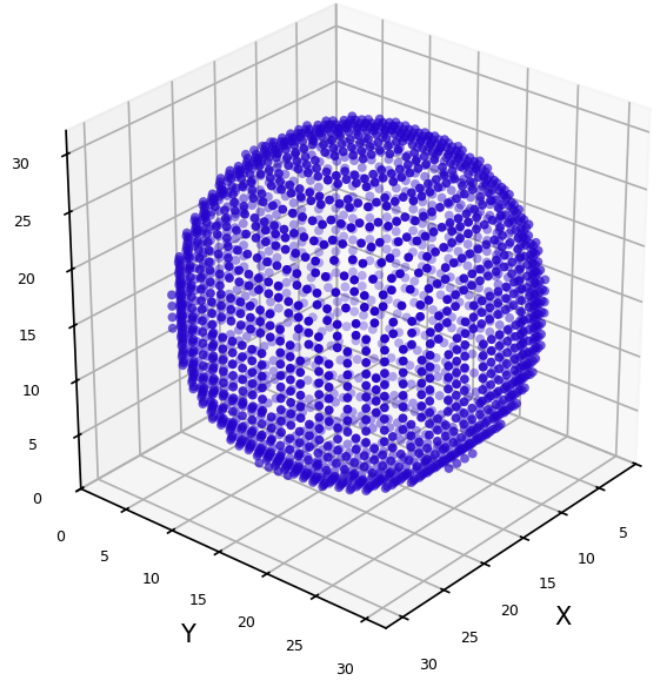


Figure 8.1: Symmetric target used for baseline morphogenesis.

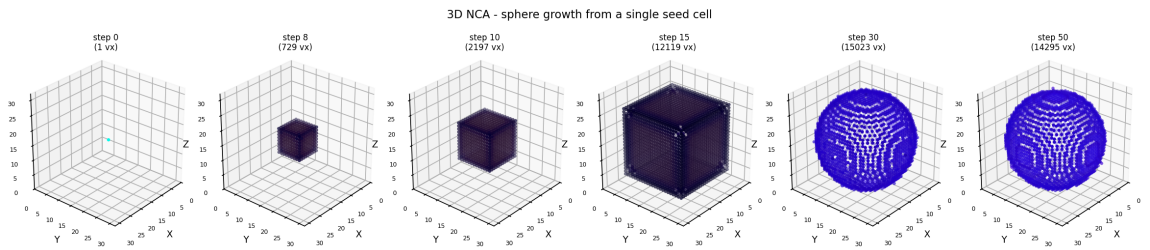


Figure 8.2: Symmetric morphogenesis from a single seed over inference steps.

Figure 8.2 shows the inference trajectory: growth from the single seed followed by stabilisation. Stability beyond the training horizon is important for interactive use, where the automaton may be stepped further than it was trained to go.

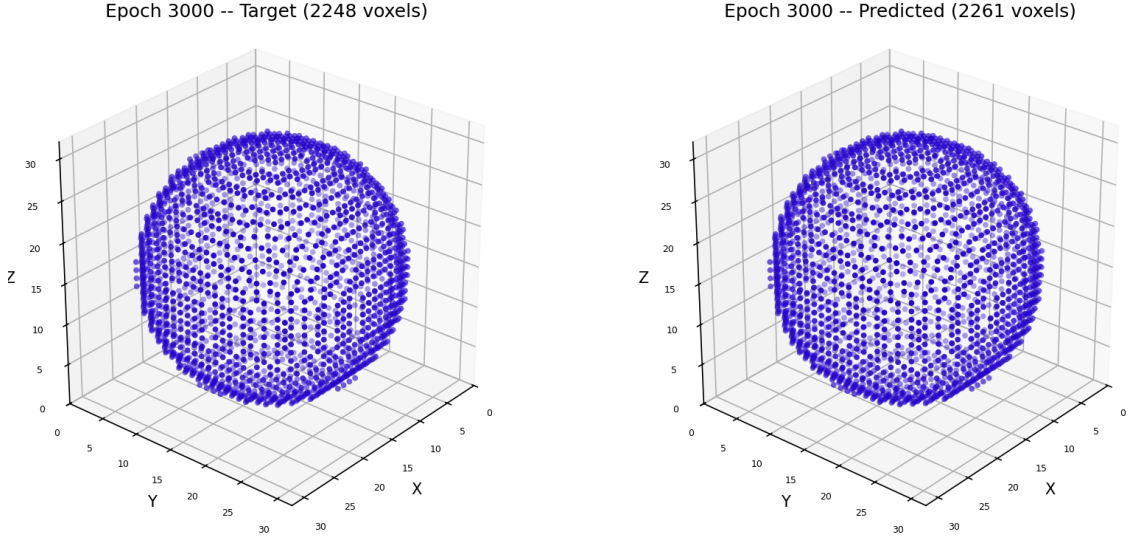


Figure 8.3: Final state at epoch 3000 compared with the symmetric target.

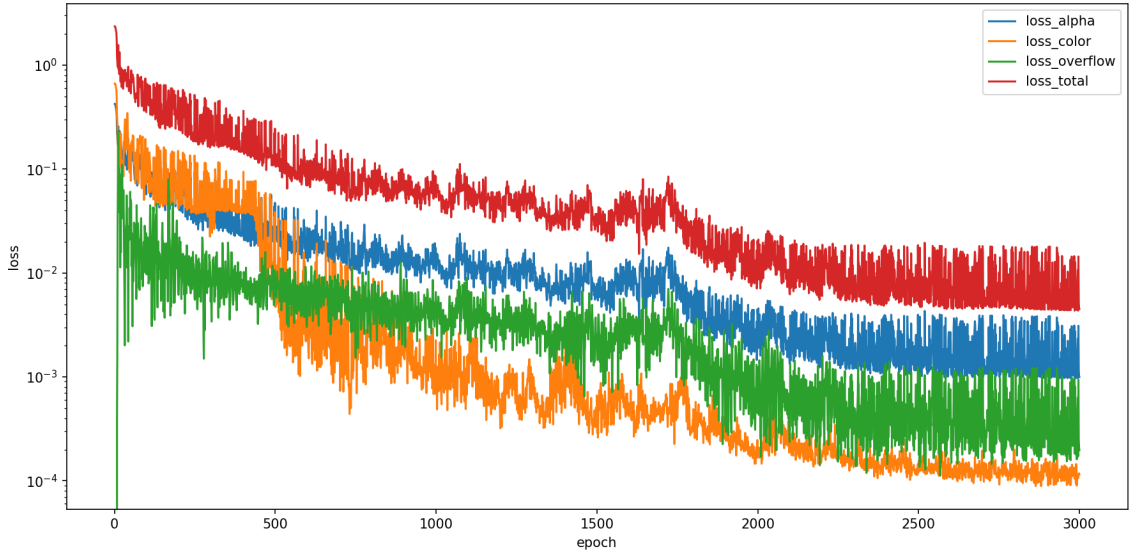


Figure 8.4: Loss curves for symmetric morphogenesis training.

Figure 8.3 compares the final output to the target and Figure 8.4 shows the loss curves. `loss_total` decreased from approximately 2.36 at the start to approximately 4.5×10^{-3} at epoch 3000, and the generated structure remained stable in post-training inference.

8.1.2 Asymmetrical Morphogenesis

The second morphogenesis experiment targeted a voxelised chess pawn and exposed a failure mode in the baseline configuration.

The first attempt reused the settings from the symmetric experiment (`pos_channels=0`, `perception.channel_groups=3`). With only isotropic local features, the model repeatedly drifted toward symmetric intermediate solutions and failed to preserve the directional structure of the target.

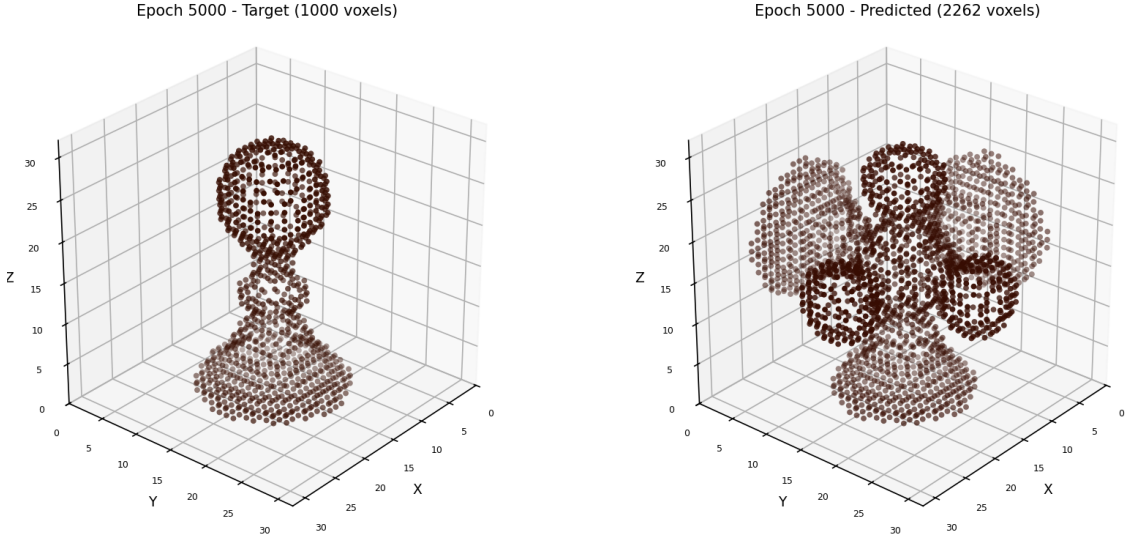


Figure 8.5: Failed asymmetric pawn run at epoch 5000 with symmetric perception configuration. The prediction overgrows the target and collapses into repeated symmetric structures.

The failure is consistent with the information available to the update rule. The three-group perception produces only isotropic neighbourhood statistics (identity, neighbour sum and Laplacian), which provides no signed cue for global orientation. Without a way to distinguish up from down or front from back, the local rule has no basis for producing directional structure.

This observation directly motivated two additions to the framework. First, `perception.channel_groups=5` was introduced, replacing the Laplacian with signed gradients along x , y and z . Second, `pos_channels=3` was added, injecting normalised absolute coordinates as read-only channels that give each cell a stable global reference. Table 8.3 lists the changes relative to the symmetric baseline.

Setting	Value
<code>cell.pos_channels</code>	3
<code>perception.channel_groups</code>	5
<code>training.num_epochs</code>	4000

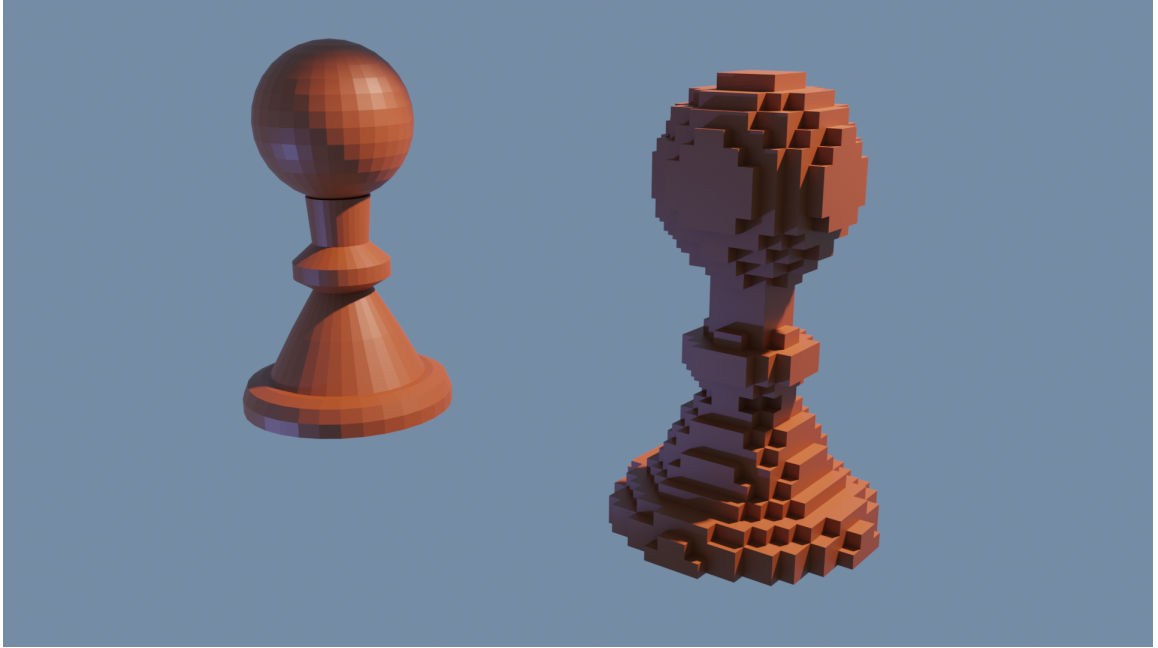
Table 8.3: Asymmetric morphogenesis changes relative to the symmetric baseline.

All other settings match Table 8.2 and Table 8.1.

Because the core model is modular and channel layout is entirely configuration-driven, applying both fixes required only config changes. No server messages, protocol definitions

or Blender UI code needed to change. This is the direct benefit of the component separation described in Section 5.3: experimenting with the model structure stays local to the model and does not affect anything outside the model.

This experiment was run in the connectionless pattern: **MorphRunner** was used directly in a notebook without a server session. Blender was used for target preparation and for final qualitative inspection of the trained model, but the training itself ran entirely within the notebook. This demonstrates that a researcher who does not need live visualisation can use the core package as a self-contained training library.



Target before training

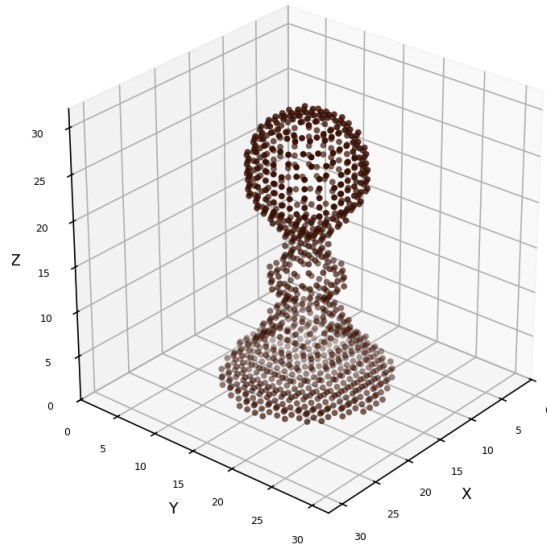


Figure 8.6: Asymmetric target prepared in Blender (top) and voxel target used during training (bottom).

Figure 8.6 illustrates an additional difficulty of mesh-based targets: thin parts can become sparse or disconnected after voxelisation, which makes them harder to grow and maintain under purely local updates.

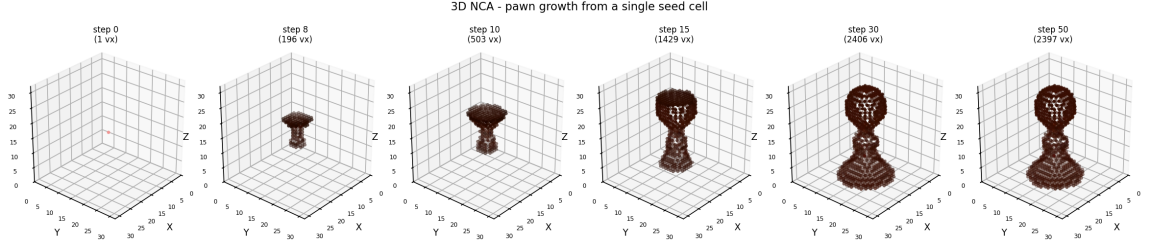


Figure 8.7: Asymmetric morphogenesis from a single seed over inference steps.

With directional gradients and positional channels enabled, the model preserves orientation throughout the inference rollout (Figure 8.7).

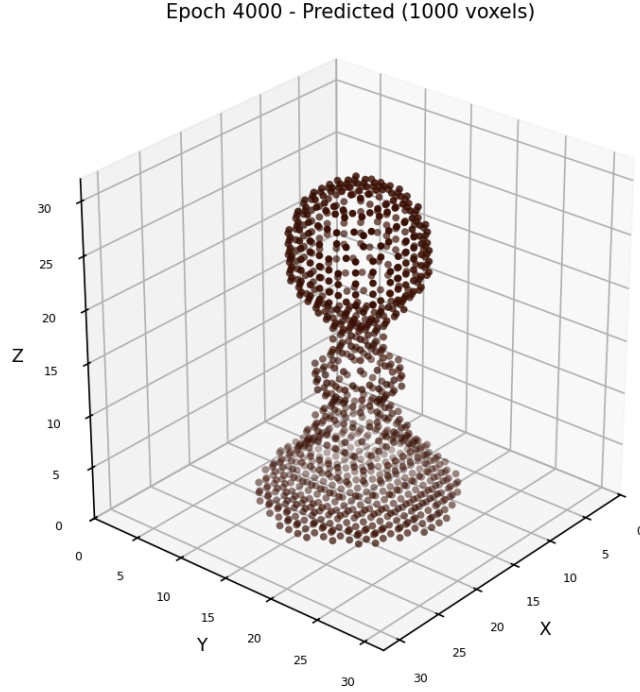


Figure 8.8: Final asymmetric output at epoch 4000.

The run reached a `loss_total` of approximately 2.75×10^{-4} at epoch 4000 while preserving the global orientation of the target. Compared to the symmetric baseline, convergence was slower and more sensitive to configuration, but the final shape fidelity was higher for this asymmetric object.

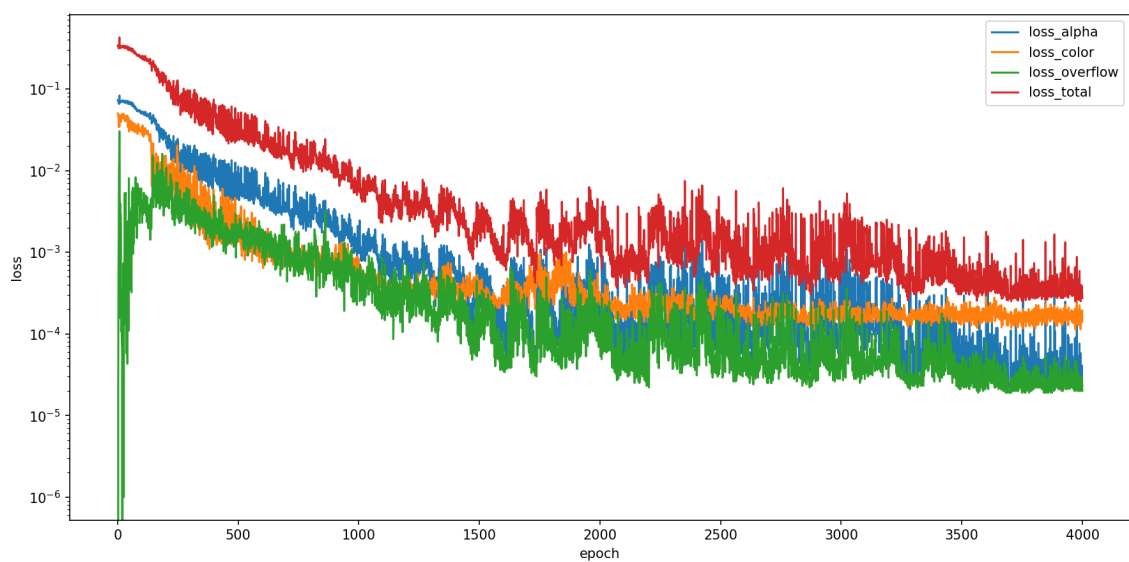


Figure 8.9: Loss curves for asymmetric morphogenesis training.

8.2 Metamorphogenesis

Multi-target conditioning in neural cellular automata has precedent in the literature. Palm et al. [16] proposed Variational Neural Cellular Automata (VNCA), which conditions a single NCA on a latent code to produce different stable forms and demonstrates recovery from significant damage. The same principle applies here: a single local update rule can encode multiple behaviours, and the active behaviour can be selected through additional channels. Sakour [20] published a related approach to conditional morphogenesis in 2D NCA during the writing of this thesis.

This work extends that idea into 3D with a distinct training algorithm and a term to describe the regime. In this thesis, *metamorphogenesis* denotes a multi-target setting in which one model both generates form and transitions between forms through task-channel conditioning. The term is formed from the Latinised scientific terms *metamorphosis* (change of form) and *morphogenesis* (generation of form). Conceptually, it extends standard morphogenesis from a single seed-to-target mapping to guided stage-to-stage development, similar to biological organisms that pass through distinct life stages such as larva, pupa and adult. Unlike the latent-code and 2D approaches in the literature, the implementation here is fully 3D, integrates with the framework’s pool-based training and schedule system, and is designed so that the runner can be connected to the server and Blender with minimal code changes.

8.2.1 Training algorithm

The key idea is a shared replay pool that holds states from all active tasks simultaneously. Each training batch is sampled from the pool and each element is supervised against the target corresponding to its stored task ID. At phase boundaries the pool is *expanded*: the best-converged predecessor states, selected by lowest MSE against the predecessor target, are copied and relabeled to the next task ID. This seeds the new task with realistic intermediate forms rather than fresh seeds, giving the transition a natural starting point. The task-channel embedding is a one-hot vector broadcast across all cells; overwriting it at inference time is the only operation needed to trigger a transition.

The training dynamics differ from single-target morphogenesis in several respects. The pool must be large enough to hold diverse states across all active tasks. Each phase boundary causes a sharp loss increase as the new target enters the batch distribution, visible as spikes in the loss curves. Loss weights and learning rate typically need per-phase adjustment because later phases contain finer structures and stronger colour discontinuities than earlier ones.

Table 8.4 lists the concrete hyperparameters for all metamorphogenesis runs.

Run	Epochs	Phases	Bounds	h	Pool	LR Schedule	Loss Wts.
Synthetic	9k	3/3/3 (k)	3k, 6k	96	32	Cosine ($2e^{-3}$)	4:1:2
Creature R1	9k	3/3/3 (k)	3k, 6k	96	32	Cosine ($2e^{-3}$)	4:1:2
Creature R2	12k	2/5/5 (k)	2k, 7k	128	48	$2e^{-3} \rightarrow 3e^{-4}$	3:2:3+ev
Creature R3	19.5k	1.5/6/12 (k)	1.5k, 7.5k	128	64	$2e^{-3} \rightarrow 1e^{-4}$	3:2:3+ev

Table 8.4: Configuration summary for Metamorphogenesis runs. Phases and Bounds denote stage durations and transition indices (in thousands, k). h is the `update.hidden_dim` and Pool is the seed buffer size. LR Schedule shows the decay trend, and Loss Wts. indicates component ratios (+ev includes event-based loss).

All runs are reproducible from the provided notebooks. For Colab, the only required setup is uploading target volumes to the expected Google Drive directory. For Kaggle, targets can be loaded from a private dataset or from the linked public datasets. For long runs Kaggle is preferred because jobs continue in the background after the browser session ends.

8.2.2 Synthetic Three-Target Curriculum

The first metamorphogenesis experiment used three analytically defined targets on a phased schedule:

- phase 1 (epochs 1–3000): seed $\rightarrow$ tetrahedron,
- phase 2 (epochs 3001–6000): tetrahedron $\rightarrow$ cube,
- phase 3 (epochs 6001–9000): cube $\rightarrow$ sphere.

The model used three task channels, three positional channels and the directional five-group perception operator. Pool expansion and relabeling were applied at phase boundaries so each new task received realistic morphing inputs rather than fresh seeds.

This run used the core-only pattern: **Grid3D** was constructed directly with a custom training loop and no **NCARunner** inheritance. **NCALogger** was used standalone for checkpointing and loss logging, with no server required. This demonstrates that the logging and training infrastructure works independently of the runner interface. The run was executed on Kaggle and took 4 hours and 1 minute. The notebook is `notebooks/kaggle_run_moprh.ipynb`.

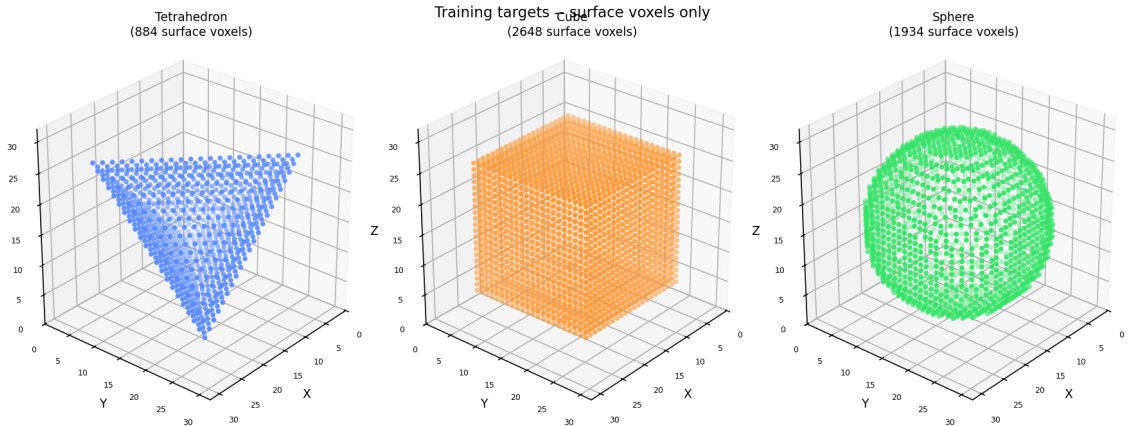


Figure 8.10: Synthetic targets for the three-target curriculum: tetrahedron, cube and sphere.

Figure 8.11 shows the characteristic pattern of the metamorphogenesis training regime: each phase boundary causes an immediate loss spike followed by gradual re-convergence, confirming that the model adjusts to each new task while retaining the previous ones. Within each phase the curve follows the same steep initial fall seen in single-target morphogenesis.

Figure 8.12 shows the intended behaviour: overwriting only the task-channel embedding is sufficient to transition the same state toward the next target, with no re-seeding required.

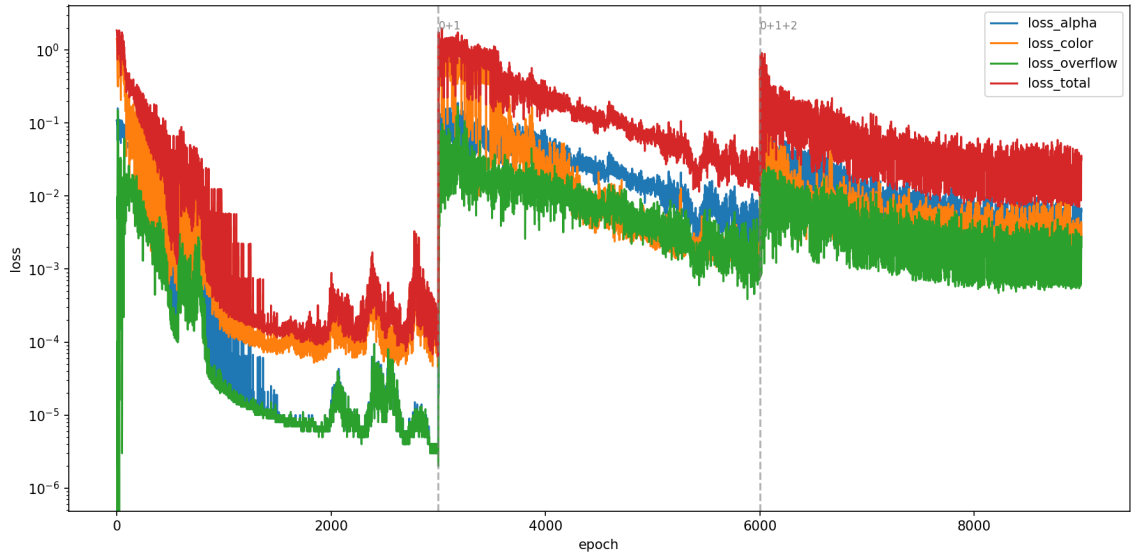


Figure 8.11: Loss curves for the synthetic curriculum. Dashed vertical lines mark phase boundaries at epochs 3000 and 6000.

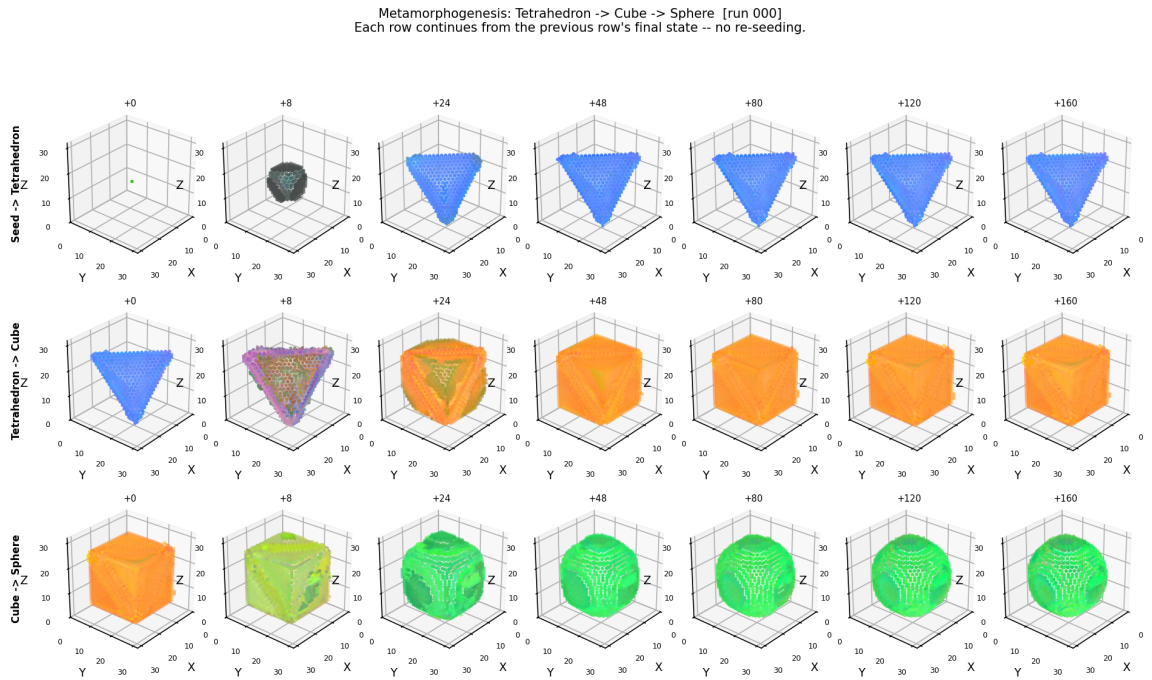


Figure 8.12: Inference progression for the synthetic curriculum. Each row continues from the previous row's final state with no re-seeding – only the task-channel embedding is overwritten.

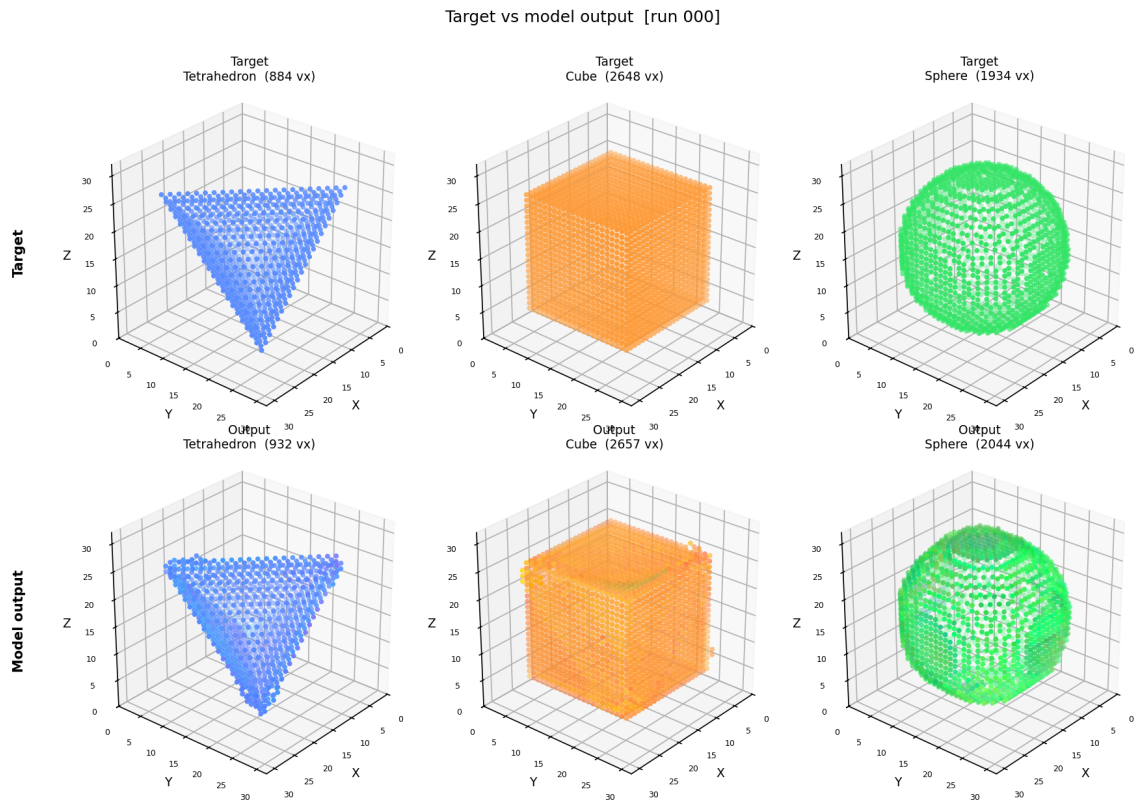


Figure 8.13: Target versus model output after synthetic curriculum training.

Figure 8.13 shows the final reconstructions. Shape fidelity is high for the sphere and cube; the tetrahedron is slightly less precise at its sharp edges but remains well-formed. The conditioning mechanism works on clean analytic targets. The result gives a clean baseline before moving to mesh-based targets.

8.2.3 Pokémon-Inspired Creature Sequence – Round 1

The second metamorphogenesis experiment applied the same phased curriculum to a more complex three-object sequence exported from Blender: Creature 1 $\rightarrow$ Creature 2 $\rightarrow$ Creature 3. The three targets are Pokémon-inspired character models (Charmander, Charmeleon and Charizard) voxelised to the same 32^3 grid.

Like the synthetic run, this experiment used the core-only pattern: `Grid3D` was constructed directly with a custom loop and no `NCARunner` inheritance. The same hyperparameters as the synthetic curriculum were used (Table 8.4, Creature R1). The run was executed on a Kaggle T4 GPU and took 4 hours and 1 minute. The notebook is `notebooks/kaggle_run_creature.ipynb`.

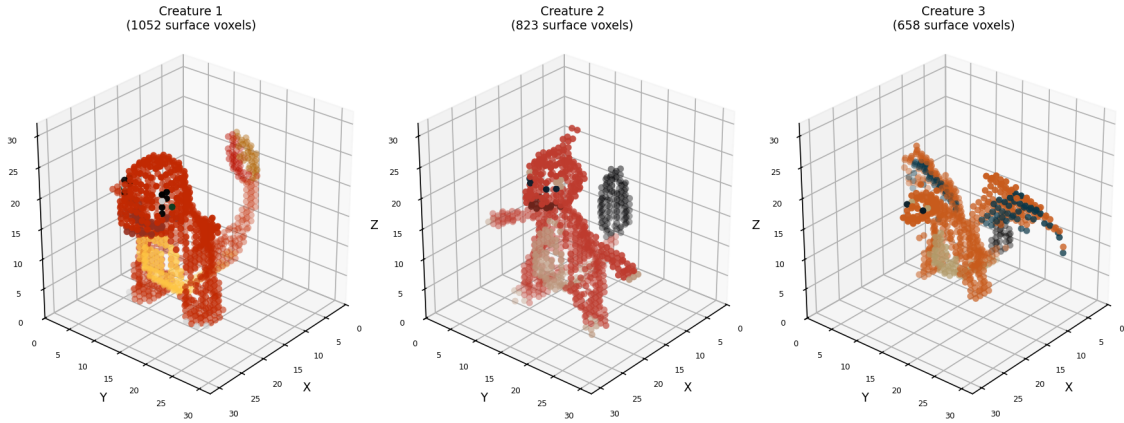


Figure 8.14: Blender-exported targets for the creature sequence: Creature 1, Creature 2 and Creature 3.

Compared to the synthetic targets, the creature models contain thinner limbs, asymmetric silhouettes and stronger colour discontinuities (Figure 8.14), all of which make training less stable.

Figure 8.15 shows the same characteristic boundary pattern as the synthetic run: each phase boundary causes a sudden loss increase followed by gradual re-convergence. The main difference is that the colour loss in phases 2 and 3 stabilises at a higher floor, reflecting the greater colour complexity of the creature targets.

Figure 8.16 shows that the conditioning mechanism transfers from synthetic to mesh-based targets. Reconstruction quality in thin or highly asymmetric regions was limited, however. Colour accuracy in the late-phase targets was worse than expected, and some limb voxels were unstable during inference. These observations led directly to three changes tested in Rounds 2 and 3: higher MLP capacity (`update.hidden_dim=128`), a larger pool and explicit per-phase weight and learning-rate schedule events.

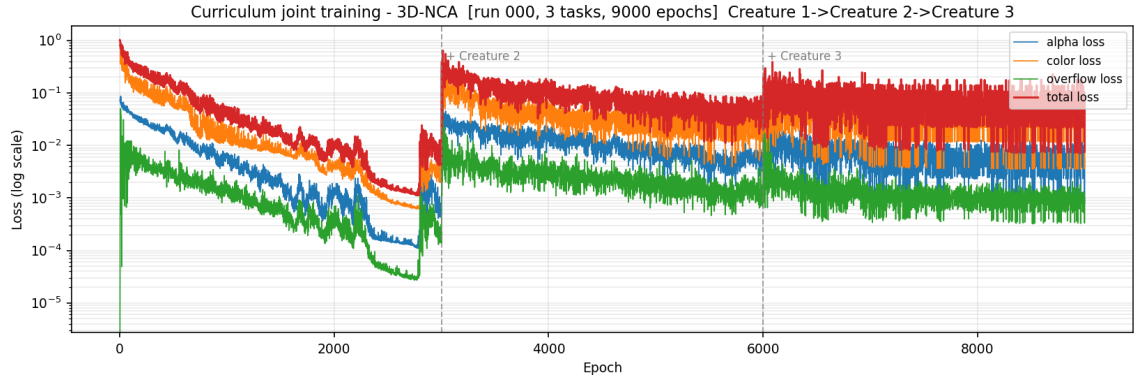


Figure 8.15: Loss curves for Creature Round 1 (9k epochs). Dashed lines mark phase boundaries at epochs 3000 and 6000.

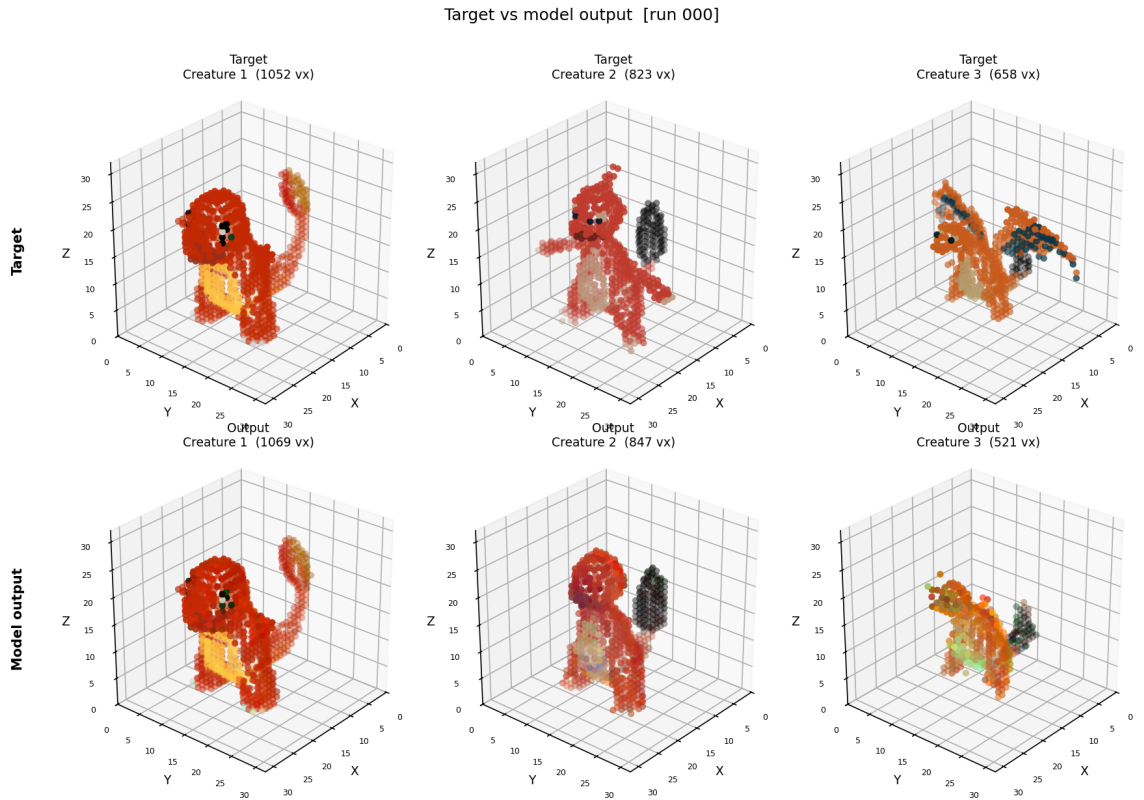


Figure 8.16: Target versus model output for Creature Round 1.

8.2.4 Pokémon-Inspired Creature Sequence – Rounds 2 and 3

Rounds 2 and 3 share the same training algorithm but differ in phase lengths and schedule parameters. Both use a `MetamorphosisRunner` that subclasses `NCARunner` and a `Schedule` populated with `Event` objects, so phase transitions, weight changes and learning rate drops all fire at specified epochs with no hardcoded epoch checks in the training loop. The pool expansion events use a custom string-backed event type (`POOL_EXPAND`) rather than a built-in `EventType`, which shows that the schedule system accepts custom event types alongside the built-in ones. Because `MetamorphosisRunner` implements the full `NCARunner` interface, connecting it to the server and Blender add-on would require only a few lines of setup code, as shown in the Implementation chapter.

Both rounds increase `update.hidden_dim` from 96 to 128 and raise the pool size (to 48 and 64 respectively) to hold more diverse states across tasks. Initial loss weights are set to $w_\alpha : w_c : w_o = 3:2:3$, with a higher colour and overflow weighting than Round 1, based on the observation that the colour loss plateau was the main limitation in Round 1.

Round 2 uses phase lengths of 2000, 5000 and 5000 epochs (boundaries at epochs 2001 and 7001). The schedule events are listed in Table 8.5. The run took 8 hours and 46 minutes on a Kaggle T4 GPU.

Epoch	Event
2001	pool expansion (task 0 $\rightarrow$ task 1)
2001	$\text{LR} \leftarrow 8 \times 10^{-4}$
2001	$w_c \leftarrow 3.0$
3501	$w_o \leftarrow 4.0$
7001	pool expansion (task 1 $\rightarrow$ task 2)
7001	$\text{LR} \leftarrow 3 \times 10^{-4}$
7001	$w_c \leftarrow 5.0$
9001	$w_o \leftarrow 6.0$

Table 8.5: Round 2 schedule events.

Round 3 uses phase lengths of 1500, 6000 and 12000 epochs (boundaries at epochs 1501 and 7501). Extending phase 2 and especially phase 3 was the primary change from Round 2, motivated by the observation that the final-phase colour loss had not fully converged within Round 2’s 5000-epoch phase 3. Two additional late-epoch events were added at epoch 15000 to push the overflow weight higher and reduce the learning rate further. The schedule events are listed in Table 8.6. The run took 11 hours and 43 minutes on a Kaggle T4 GPU.

Epoch	Event
1501	pool expansion (task 0 $\rightarrow$ task 1)
1501	$\text{LR} \leftarrow 8 \times 10^{-4}$
1501	$w_c \leftarrow 3.0$
3501	$w_o \leftarrow 4.0$
7501	pool expansion (task 1 $\rightarrow$ task 2)
7501	$\text{LR} \leftarrow 3 \times 10^{-4}$
7501	$w_c \leftarrow 5.0$
15000	$\text{LR} \leftarrow 1 \times 10^{-4}$
15000	$w_o \leftarrow 8.0$

Table 8.6: Round 3 schedule events.

Both notebooks are `notebooks/kaggle_run_creature_final.ipynb`.

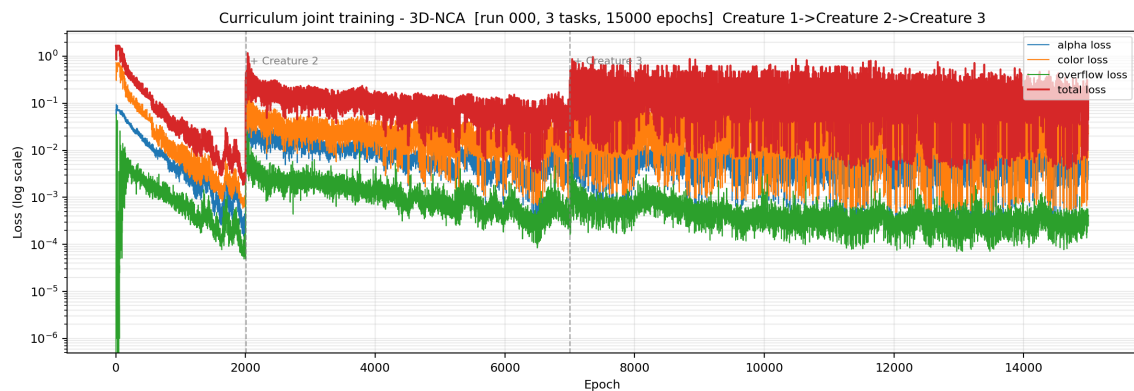


Figure 8.17: Loss curves for Creature Round 2 (12k epochs). Phase boundaries at epochs 2001 and 7001.

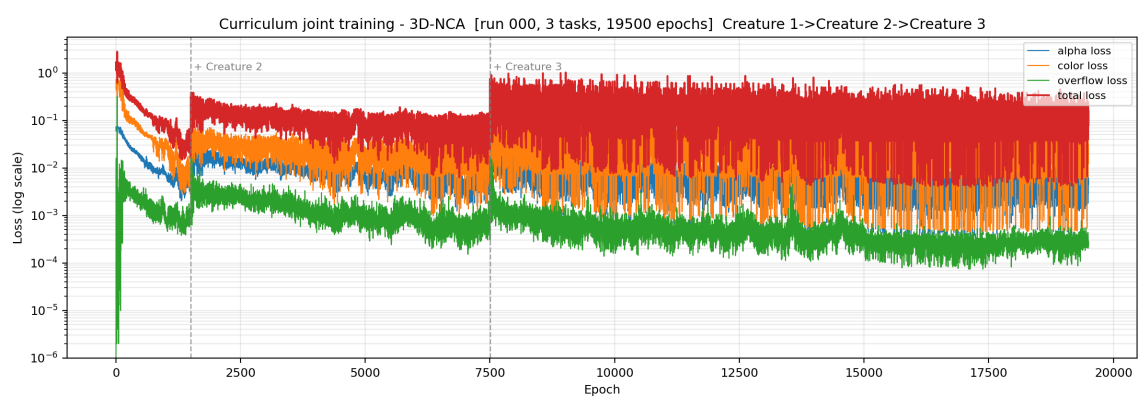


Figure 8.18: Loss curves for Creature Round 3 (19.5k epochs). Phase boundaries at epochs 1501 and 7501.

Figures 8.17 and 8.18 show the loss curves for both rounds. Both exhibit the same boundary spikes seen in all previous metamorphogenesis runs. Compared to Round 2, Round 3 shows continued colour loss improvement throughout the extended phase 3. The late-epoch learning rate reduction and increased overflow weight at epoch 15000 visibly reduce the total loss in the final training segment.

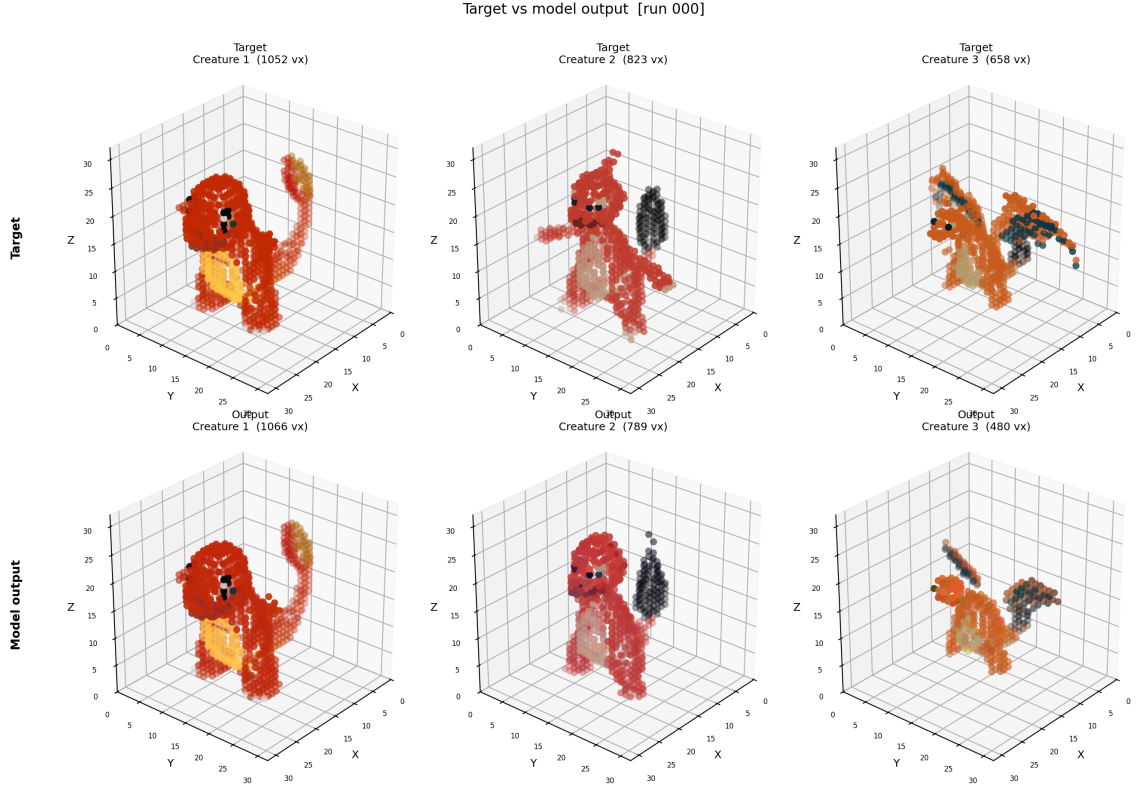


Figure 8.19: Target versus model output for Creature Round 2 (12k epochs).

Figures 8.19 and 8.20 show the target versus output comparisons. Round 2 improved colour reconstruction in phases 1 and 2 relative to Round 1, but phase 3 still showed colour artefacts in thin wing regions. Round 3 reduced these further: the extended phase 3 and the final overflow weight adjustment cleared most stray floating voxels and improved colour coherence across all three tasks, although thin structures remained the hardest part to reconstruct at the 32^3 grid resolution.

Figure 8.21 shows the Round 3 inference trajectory. The model transitions continuously between creatures without re-seeding and the intermediate states show a gradual change in form rather than an abrupt switch.

8.2.5 Summary

Table 8.7 compares the four metamorphogenesis runs.

The progression from Round 1 to Round 3 shows a clear trend: higher model capacity, a larger pool and explicit per-phase scheduling consistently improved reconstruction quality. The most persistent difficulty across all runs was thin or highly asymmetric voxel regions, where the 32^3 grid resolution is a hard constraint on the level of detail achievable.

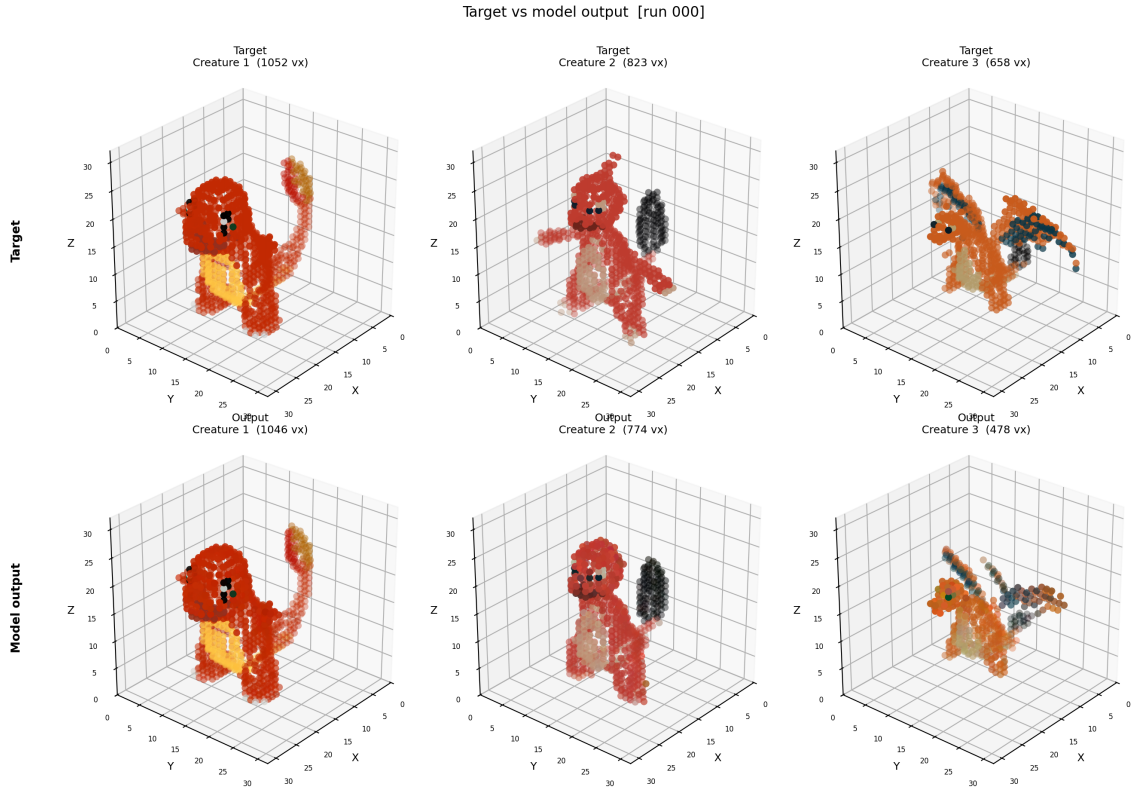


Figure 8.20: Target versus model output for Creature Round 3 (19.5k epochs).

Run	Epochs	h	Pool	Runtime	Main outcome
Synthetic	9000	96	32	4 h 01 min	Conditioning validated on analytic targets
Creature R1	9000	96	32	4 h 01 min	Conditioning transfers to mesh targets; colour quality limited
Creature R2	12000	128	48	8 h 46 min	Colour improved; phase 3 not fully converged
Creature R3	19500	128	64	11 h 43 min	Best overall fidelity; thin structures remain challenging

Table 8.7: Metamorphogenesis run summary.

Metamorphogenesis: Creature 1->Creature 2->Creature 3 [run 000]

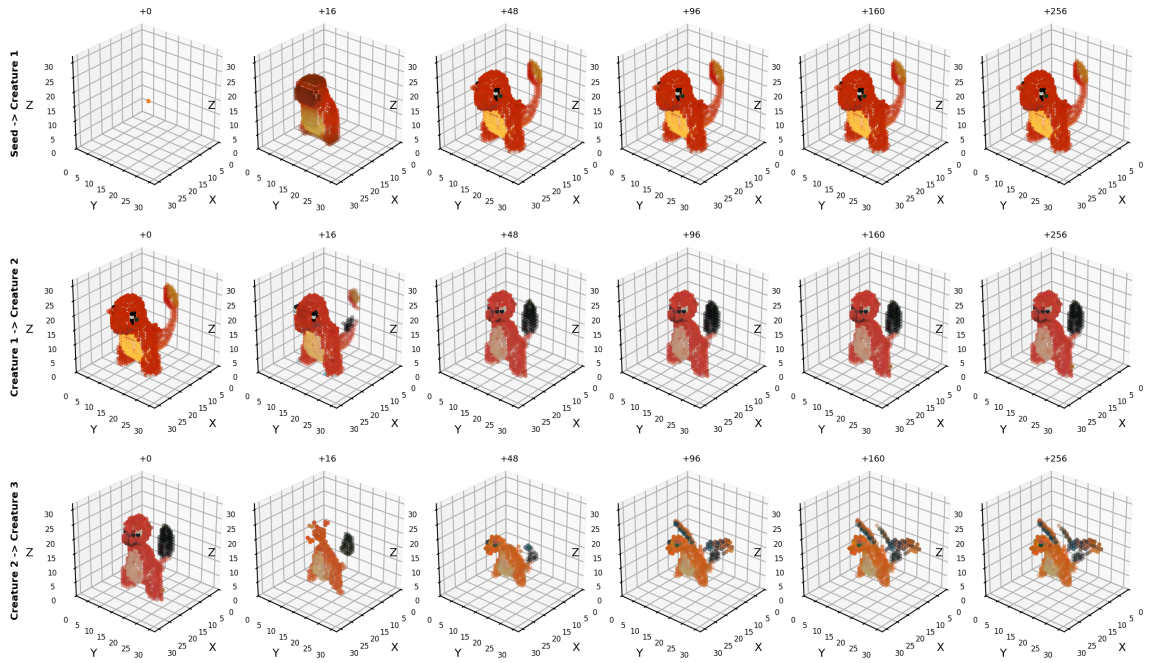


Figure 8.21: Inference progression for Creature Round 3. Each row continues from the previous row's final state; only the task-channel embedding is overwritten between rows.

Chapter 9

Conclusion

The goal of this thesis was to design and implement a modular framework for interactive experimentation with three-dimensional neural cellular automata. The framework connects mesh-based target preparation, TCP-mediated GPU training and live Blender viewport rendering into a single workflow. All design goals were met: the Blender add-on provides mesh voxelisation with per-voxel colour sampling from Blender materials and supports direct transmission of targets to the training server; the TCP server exposes any conforming backend over a local or tunnelled connection, including remote sessions via ngrok; and the standalone Python core package works from scripts and Jupyter notebooks without Blender.

The framework was validated through three categories of experiments. Symmetric morphogenesis confirmed stable growth from a single seed, with total loss converging from 2.36 to 4.5×10^{-3} over 3 000 epochs. Asymmetric morphogenesis revealed a symmetry problem in the baseline configuration, which was resolved by enabling directional gradient filters and positional channels; the corrected run reached a total loss of 2.75×10^{-4} at epoch 4 000. Multi-target metamorphogenesis confirmed that a single model can grow and transition between three distinct voxel targets through task-channel conditioning without re-seeding, with the best creature run completing 19 500 epochs of training. The extensibility of the framework was validated directly by the metamorphogenesis experiment: a custom `NCARunner` subclass was sufficient to introduce the new training regime with no changes to the server, protocol or visualisation pipeline.

Several directions are open for future development. In the short term, the framework could be extended with additional training regimes – specifically persistence and full re-generation – exposed through the existing runner interface and demonstrated interactively through tools such as a Blender viewport eraser that lets the user remove parts of a grown structure and observe live recovery. The voxelisation pipeline could also be extended to encode additional Blender data into training targets, such as armature rigs or shape keys, with the goal of producing models whose output retains rig information and supports animation-compatible workflows. In the longer term, scaling to higher grid resolutions such as 64^3 or 128^3 would require replacing the current dense tensor representation with sparse data structures, making the approach practical for large volumetric domains where most voxels are empty.

Bibliography

- [1] BERLEKAMP, E. R.; CONWAY, J. H. and GUY, R. K. *Winning Ways for Your Mathematical Plays, Volume 2*. Academic Press, 1982.
- [2] CHAN, B. W.-C. Lenia: Biology of Artificial Life. *Complex Systems*, 2019.
- [3] COMMUNITY, P. *PyTorch Documentation*. 2025. Available at: <https://docs.pytorch.org/stable/index.html>.
- [4] COOK, M. Universality in Elementary Cellular Automata. *Complex Systems*, 2004.
- [5] EDDY, W. *Transmission Control Protocol (TCP)* RFC 9293. 2022. Available at: <https://doi.org/10.17487/RFC9293>.
- [6] GARDNER, M. Mathematical Games: The Fantastic Combinations of John Conway’s New Solitaire Game „Life“. *Scientific American*, 1970. Available at: <https://web.stanford.edu/class/sts145/Library/life.pdf>.
- [7] HORVÁTH, D. and LEVIN, M. Self-Classifying MNCA. *Distill*, 2021. Available at: <https://distill.pub/2021/self-classifying-mnca/>.
- [8] ILACHINSKI, A. *Cellular Automata: A Discrete Universe*. World Scientific, 2001.
- [9] JHA, A. *3D Growing Neural Cellular Automata*. 2024. Available at: <https://github.com/Aadityaza/3d-Growing-neural-cellular-automata>. GitHub repository.
- [10] KINDEREN, E. de and WUENSCHÉ, A. 3D CA in a Fluid Dynamics PDE. In: *Proceedings of the Second International Conference on Evolvable Systems*. Springer, 1998.
- [11] LAINE, S. and KARRAS, T. Efficient Sparse Voxel Octrees. In: *Proceedings of the 2010 ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*. New York, NY, USA: ACM, 2010. Available at: <https://doi.org/10.1145/1730804.1730814>.
- [12] MARINESCU, D. C. Nature-Inspired Algorithms and Systems. In: *Complex Systems and Clouds*. Elsevier, 2017, p. 33–63. Available at: <https://www.sciencedirect.com/science/article/pii/B9780128040416000025>.
- [13] MORDVINTSEV, A.; RANDAZZO, E.; NIKLASSON, E. and LEVIN, M. Growing Neural Cellular Automata. *Distill*, 2020. Available at: <https://distill.pub/2020/growing-ca/>.

- [14] MUSETH, K. VDB: High-Resolution Sparse Volumes with Dynamic Topology. *ACM Transactions on Graphics*, 2013. Available at: <https://doi.org/10.1145/2487228.2487235>.
- [15] NEUMANN, J. von. *Theory of Self-Reproducing Automata*. University of Illinois Press, 1966.
- [16] PALM, R. B.; GONZÁLEZ DUQUE, M.; SUDHAKARAN, S. and RISI, S. *Variational Neural Cellular Automata*. 2022. Available at: <https://arxiv.org/abs/2201.12360>. ICLR 2022.
- [17] PASZKE, A.; GROSS, S.; MASSA, F. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: *NeurIPS*. 2019. Available at: <https://arxiv.org/abs/1912.01703>.
- [18] RAFLER, S. *Generalization of Conway’s Game of Life to a Continuous Domain – SmoothLife*. 2011. Available at: <https://arxiv.org/abs/1111.1567>.
- [19] ROSSI, F. and ORSINI, M. *3D Growing Neural Cellular Automaton*. 2021. Available at: <https://github.com/SkyLionx/3d-cellular-automaton>. GitHub repository.
- [20] SAKOUR, A. *Conditional Morphogenesis: Emergent Generation of Structural Digits via Neural Cellular Automata*. 2025. Available at: <https://arxiv.org/abs/2512.08360>.
- [21] SUDHAKARAN, S.; GRBIC, D.; LI, S.; KATONA, A.; NAJARRO, E. et al. Growing 3D Artefacts and Functional Machines with Neural Cellular Automata. In: *Proceedings of the Artificial Life Conference*. 2021. Available at: https://doi.org/10.1162/isal_a_00451.
- [22] TOBIAS, P. A. G. *3D Cellular Automata*. 2009. Available at: <https://eprints.bournemouth.ac.uk/17408/>.
- [23] TURING, A. M. The Chemical Basis of Morphogenesis. *Philosophical Transactions of the Royal Society of London. Series B, Biological Sciences*, 1952, vol. 237, no. 641, p. 37–72.
- [24] WOLFRAM, S. Statistical Mechanics of Cellular Automata. *Reviews of Modern Physics*, 1983.
- [25] WOLFRAM, S. *A New Kind of Science*. Wolfram Media, 2002. ISBN 1-57955-008-8. Available at: <https://www.wolframscience.com/nks/>.